CrewAI Guide (official docs)

Contents

Introduction **3**

What is CrewAI? **3**
Watch: Building CrewAI Agents & Flows with Coding Agent Skills . . . 3
The CrewAI Architecture . . . 3
1. Flows: The Backbone . . . 4
2. Crews: The Intelligence . . . 4
How It All Works Together . . . 6
Key Features . . . 6
When to Use Crews vs. Flows . . . 6
Why Choose CrewAI? . . . 6
Ready to Start Building? . . . 6

Agents **8**
Overview of an Agent . . . 8
Agent Attributes . . . 9
Creating Agents . . . 10
JSONC Configuration (Recommended) . . . 10
Classic YAML Configuration . . . 11
Direct Code Definition . . . 11
Parameter Details . . . 14
Agent Tools . . . 14
Agent Memory and Context . . . 15
Context Window Management . . . 15
How Context Window Management Works . . . 15
Automatic Context Handling (`respect_context_window=True`) . . . 15
Strict Context Limits (`respect_context_window=False`) . . . 16
Choosing the Right Setting . . . 16
Alternative Approaches for Large Data . . . 17
Context Window Best Practices . . . 17
Troubleshooting Context Issues . . . 17
Direct Agent Interaction with `kickoff()` . . . 18
How `kickoff()` Works . . . 18
Parameters and Return Values . . . 18
Structured Output . . . 19
Multiple Messages . . . 19
Async Support . . . 19
Upgrading Agents from Earlier Releases . . . 19
`CrewAgentExecutor` Is Removed . . . 20
Oversized Tool Results Are Deferred . . . 20
Context Is Summarized Before It Overflows . . . 20
Repeated Tool Calls Stop the Task . . . 20
How the Context Window Size Is Chosen . . . 21
Important Considerations and Best Practices . . . 21

Security and Code Execution . . . 21
Performance Optimization . . . 21
Memory and Context Management . . . 21
Advanced Features . . . 21
Agent Collaboration . . . 21
Date Awareness and Reasoning . . . 21
Model Compatibility . . . 22
Troubleshooting Common Issues . . . 22

Crews **23**
Overview . . . 23
Crew Attributes . . . 23
Creating Crews . . . 25
JSONC Configuration (Recommended) . . . 26
Classic Python/YAML Configuration . . . 27
Direct Code Definition (Alternative) . . . 27
Crew Output . . . 28
Crew Output Attributes . . . 28
Crew Output Methods and Properties . . . 28
Accessing Crew Outputs . . . 28
Accessing Crew Logs . . . 29
Checkpointing . . . 29
Quick Start . . . 29
Full Control with `CheckpointConfig` . . . 29
Resuming from a Checkpoint . . . 30
`CheckpointConfig` Attributes . . . 30
Memory Utilization . . . 30
Cache Utilization . . . 30
Crew Usage Metrics . . . 31
Crew Execution Process . . . 31
Kicking Off a Crew . . . 31
Different Ways to Kick Off a Crew . . . 31
Streaming Crew Execution . . . 32
Replaying from a Specific Task . . . 33
Replaying from a Specific Task Using the CLI . . . 33

Flows **34**
Overview . . . 34
Getting Started . . . 34
@start() . . . 36
@listen() . . . 36
Flow Output . . . 36
Flow Usage Metrics . . . 39
UsageMetrics field semantics . . . 39
Flow State Management . . . 40
Unstructured State Management . . . 40
Structured State Management . . . 41
Choosing Between Unstructured and Structured State Management . . . 42
Flow Persistence . . . 43
Class-Level Persistence . . . 43
Method-Level Persistence . . . 43
Forking Persisted State . . . 43
How It Works . . . 44
Important Considerations . . . 44
Technical Advantages . . . 44
Flow Control . . . 45
Conditional Logic: `or` . . . 45

Conditional Logic: `and` . . . 45
Router . . . 46
Human in the Loop (human feedback) . . . 48
Adding Agents to Flows . . . 49
Adding Crews to Flows . . . 51
Folder Structure . . . 52
Building Your Crews . . . 52
Connecting Crews in `main.py` . . . 52
Running the Flow . . . 53
Plot Flows . . . 54
What are Plots? . . . 54
How to Generate a Plot . . . 54
Understanding the Plot . . . 55
Conclusion . . . 55
Next Steps . . . 55
Running Flows . . . 56
Using the Flow API . . . 56
Streaming Flow Execution . . . 56
Memory in Flows . . . 56
Built-in Methods . . . 56
Example: Research and Analyze Flow . . . 57
Using the CLI . . . 57

Introduction

Build AI agent teams that work together to tackle complex tasks

What is CrewAI?

CrewAI is the leading open-source framework for orchestrating autonomous AI agents and building complex workflows.

It empowers developers to build production-ready multi-agent systems by combining the collaborative intelligence of **Crews** with the precise control of **Flows**.

- **CrewAI Flows**: The backbone of your AI application. Flows allow you to create structured, event-driven workflows that manage state and control execution. They provide the scaffolding for your AI agents to work within.
- **CrewAI Crews**: The units of work within your Flow. Crews are teams of autonomous agents that collaborate to solve specific tasks delegated to them by the Flow.

With over 100,000 developers certified through our community courses, CrewAI is the standard for enterprise-ready AI automation.

Watch: Building CrewAI Agents & Flows with Coding Agent Skills

Install our coding agent skills (Claude Code, Codex, . . .) to quickly get your coding agents up and running with CrewAI.

You can install it with `npx skills add crewaiinc/skills`

https://www.loom.com/embed/befb9f68b81f42ad8112bfdd95a780af

The CrewAI Architecture

CrewAI's architecture is designed to balance autonomy with control.

1. **Flows: The Backbone**

   [!NOTE]

   Think of a Flow as the "manager" or the "process definition" of your application. It defines the steps, the logic, and how data moves through your system.

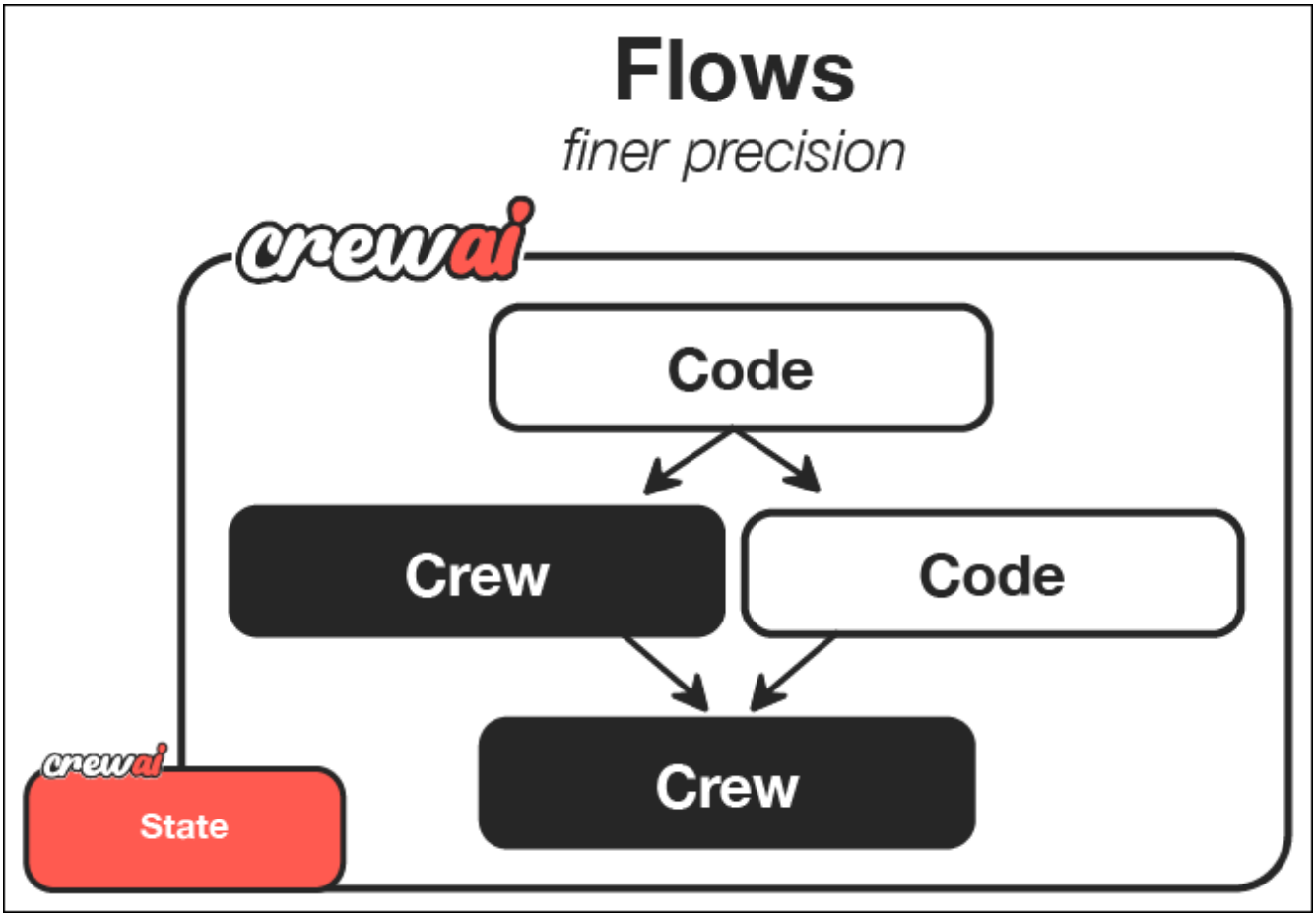


Figure 1: CrewAI Framework Overview

CrewAI Framework Overview

Flows provide:

- **State Management**: Persist data across steps and executions.
- **Event-Driven Execution**: Trigger actions based on events or external inputs.
- **Control Flow**: Use conditional logic, loops, and branching.

2. **Crews: The Intelligence**

   [!NOTE]

   Crews are the "teams" that do the heavy lifting. Within a Flow, you can trigger a Crew to tackle a complex problem requiring creativity and collaboration.

CrewAI Framework Overview

Crews provide:

- **Role-Playing Agents**: Specialized agents with specific goals and tools.
- **Autonomous Collaboration**: Agents work together to solve tasks.
- **Task Delegation**: Tasks are assigned and executed based on agent capabilities.

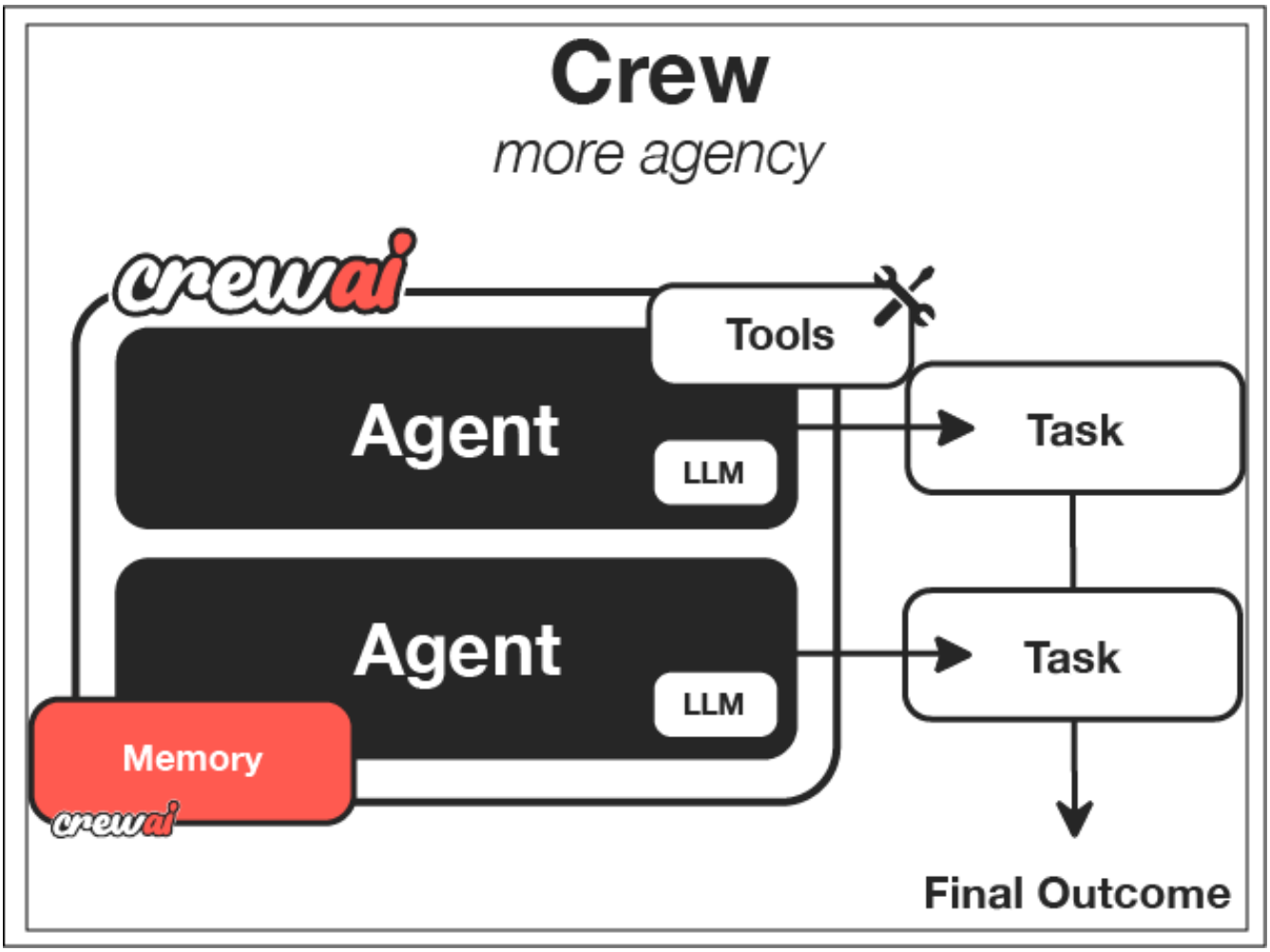


Figure 2: CrewAI Framework Overview

How It All Works Together

1. **The Flow** triggers an event or starts a process.
2. **The Flow** manages the state and decides what to do next.
3. **The Flow** delegates a complex task to a **Crew**.
4. **The Crew**'s agents collaborate to complete the task.
5. **The Crew** returns the result to the **Flow**.
6. **The Flow** continues execution based on the result.

Key Features

[card] Production-Grade Flows

Build reliable, stateful workflows that can handle long-running processes and complex logic.

[card] Autonomous Crews

Deploy teams of agents that can plan, execute, and collaborate to achieve high-level goals.

[card] Flexible Tools

Connect your agents to any API, database, or local tool.

[card] Enterprise Security

Designed with security and compliance in mind for enterprise deployments.

When to Use Crews vs. Flows

The short answer: Use both.

For any production-ready application, **start with a Flow**.

- **Use a Flow** to define the overall structure, state, and logic of your application.
- **Use a Crew** within a Flow step when you need a team of agents to perform a specific, complex task that requires autonomy.

Use Case	Architecture
Simple Automation	Single Flow with Python tasks
Complex Research	Flow managing state -> Crew performing research
Application Backend	Flow handling API requests -> Crew generating content -> Flow saving to DB

Why Choose CrewAI?

- **Autonomous Operation**: Agents make intelligent decisions based on their roles and available tools
- **Natural Interaction**: Agents communicate and collaborate like human team members
- **Extensible Design**: Easy to add new tools, roles, and capabilities
- **Production Ready**: Built for reliability and scalability in real-world applications
- **Security-Focused**: Designed with enterprise security requirements in mind
- **Cost-Efficient**: Optimized to minimize token usage and API calls

Ready to Start Building?

[card] Build Your First Flow

Learn how to create structured, event-driven workflows with precise control over execution.

[card] Build Your First Crew

Step-by-step tutorial to create a collaborative AI team that works together to solve complex problems.

[card] Install CrewAI

Get started with CrewAI in your development environment.

[card] Quick Start

Scaffold a Flow, run a crew with one agent, and generate a report end to end.

[card] Join the Community

Connect with other developers, get help, and share your CrewAI experiences.

This documentation is built and hosted on Mintlify, a developer documentation platform.

Agents

Detailed guide on creating and managing agents within the CrewAI framework.

Overview of an Agent

In the CrewAI framework, an `Agent` is an autonomous unit that can:

- Perform specific tasks
- Make decisions based on its role and goal
- Use tools to accomplish objectives
- Communicate and collaborate with other agents
- Maintain memory of interactions
- Delegate tasks when allowed

[!TIP]

Think of an agent as a specialized team member with specific skills, expertise, and responsibilities. For example, a `Researcher` agent might excel at gathering and analyzing information, while a `Writer` agent might be better at creating content.

[!NOTE]

CrewAI AMP includes a Visual Agent Builder that simplifies agent creation and configuration without writing code. Design your agents visually and test them in real-time.

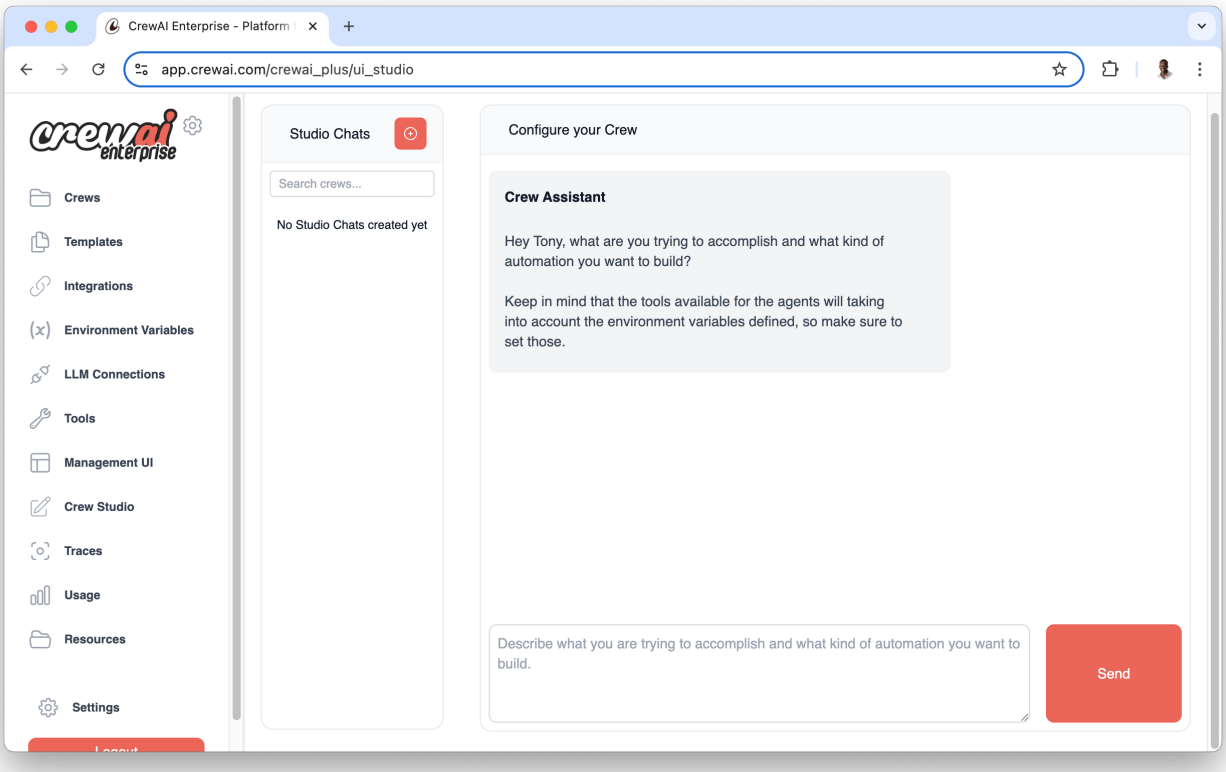


Figure 3: Visual Agent Builder Screenshot

The Visual Agent Builder enables:

- Intuitive agent configuration with form-based interfaces
- Real-time testing and validation

- Template library with pre-configured agent types
- Easy customization of agent attributes and behaviors

Agent Attributes

Attribute	Parameter	Type	Description
Role	`role`	`str`	Defines the agent's function and expertise within the crew.
Goal	`goal`	`str`	The individual objective that guides the agent's decision-making.
Backstory	`backstory`	`str`	Provides context and personality to the agent, enriching interactions.
LLM *(optional)*	`llm`	`Union[str, LLM, Any]`	Language model that powers the agent. Defaults to the model specified in `OPENAI_MODEL_NAME` or "gpt-4".
Tools *(optional)*	`tools`	`List[BaseTool]`	Capabilities or functions available to the agent. Defaults to an empty list.
Function Calling LLM *(optional)*	`function_calling_llm`	`Optional[Any]`	Language model for tool calling, overrides crew's LLM if specified.
Max Iterations *(optional)*	`max_iter`	`int`	Maximum iterations before the agent must provide its best answer. Default is 20.
Max RPM *(optional)*	`max_rpm`	`Optional[int]`	Maximum requests per minute to avoid rate limits.
Max Execution Time *(optional)*	`max_execution_time`	`Optional[int]`	Maximum time (in seconds) for task execution.
Verbose *(optional)*	`verbose`	`bool`	Enable detailed execution logs for debugging. Default is False.
Allow Delegation *(optional)*	`allow_delegation`	`bool`	Allow the agent to delegate tasks to other agents. Default is False.
Step Callback *(optional)*	`step_callback`	`Optional[Any]`	Function called after each agent step, overrides crew callback.
Cache *(optional)*	`cache`	`bool`	Enable caching for tool usage. Default is True.
System Template *(optional)*	`system_template`	`Optional[str]`	Custom system prompt template for the agent.
Prompt Template *(optional)*	`prompt_template`	`Optional[str]`	Custom prompt template for the agent.
Response Template *(optional)*	`response_template`	`Optional[str]`	Custom response template for the agent.
Allow Code Execution *(optional)*	`allow_code_execution`	`Optional[bool]`	Enable code execution for the agent. Default is False.

Attribute	Parameter	Type	Description
Max Retry Limit *(optional)*	`max_retry_limit`	`int`	Maximum number of retries when an error occurs. Default is 2.
Respect Context Window *(optional)*	`respect_context_window`	`bool`	Keep messages under context window size by summarizing. Default is True.
Use Better Compaction *(optional)*	`use_better_compaction`	`bool`	Defer oversized native tool results to temporary files the agent reads in sections. Default is True.
Code Execution Mode *(optional)*	`code_execution_mode`	`Literal["safe", "unsafe"]`	Mode for code execution: 'safe' (using Docker) or 'unsafe' (direct). Default is 'safe'.
Multimodal *(optional)*	`multimodal`	`bool`	Whether the agent supports multimodal capabilities. Default is False.
Inject Date *(optional)*	`inject_date`	`bool`	Whether to automatically inject the current date into the agent's prompt. Default is False.
Date Format *(optional)*	`date_format`	`str`	Format string for date when inject_date is enabled. Default is "%Y-%m-%d" (ISO format).
Reasoning *(optional)*	`reasoning`	`bool`	Whether the agent should reflect and create a plan before executing a task. Default is False.
Max Reasoning Attempts *(optional)*	`max_reasoning_attempts`	`Optional[int]`	Maximum number of reasoning attempts before executing the task. If None, will try until ready.
Embedder *(optional)*	`embedder`	`Optional[Dict[str, Any]]`	Configuration for the embedder used by the agent.
Knowledge Sources *(optional)*	`knowledge_sources`	`Optional[List[BaseKnowledgeSource]]`	Knowledge sources available to the agent.
Use System Prompt *(optional)*	`use_system_prompt`	`Optional[bool]`	Whether to use system prompt (for o1 model support). Default is True.

Creating Agents

There are two common ways to create agents in CrewAI: using **JSONC project configuration (recommended for new crews)** or defining them **directly in code**.

JSONC Configuration (Recommended)

New projects created with `crewai create crew <name>` use JSON-first configuration. Each agent is defined in `agents/<agent_name>.jsonc`, and `crew.jsonc` lists which agents are part of the crew.

After creating your CrewAI project as outlined in the Installation section, edit the generated files in `agents/`.

> [!NOTE]
>
> Use `{placeholder}` values in `role`, `goal`, or `backstory`. Put defaults in `crew.jsonc` under `inputs`; `crewai run` prompts for any missing values.

Here's an example `agents/researcher.jsonc` file:

```
{
  "role": "{topic} Senior Data Researcher",
  "goal": "Uncover cutting-edge developments in {topic}",
  "backstory": "You find the most relevant information and present it clearly.",
  "llm": "openai/gpt-4o",
  "tools": ["SerperDevTool"],
  "settings": {
    "verbose": true,
    "allow_delegation": false,
    "max_iter": 20
  }
}
```

Then include that agent from `crew.jsonc`:

```
{
  "name": "Research Crew",
  "agents": ["researcher"],
  "tasks": [
    {
      "name": "research_task",
      "description": "Research {topic}",
      "expected_output": "A concise briefing about {topic}",
      "agent": "researcher"
    }
  ],
  "inputs": {
    "topic": "AI Agents"
  }
}
```

Agent files support any public `Agent` field. Common fields include `role`, `goal`, `backstory`, `llm`, `tools`, `function_calling_llm`, `guardrail`, `step_callback`, and `settings`. Behavior options such as `verbose`, `allow_delegation`, `max_iter`, `max_rpm`, `memory`, `cache`, `planning_config`, and `use_system_prompt` can be placed at the top level or under `settings`; values in `settings` take precedence.

> [!NOTE]
>
> JSONC supports comments and trailing commas. If both `agents/<name>.jsonc` and `agents/<name>.json` exist, CrewAI uses the JSONC file.

Classic YAML Configuration

Classic projects created with `crewai create crew <name> --classic` use `config/agents.yaml` and a `@CrewBase` class in `crew.py`. This remains supported for teams that want Python decorators or existing YAML projects.

Direct Code Definition

You can create agents directly in code by instantiating the `Agent` class. Here's a comprehensive example showing all available parameters:

```
from crewai import Agent
from crewai_tools import SerperDevTool
```

```python
# Create an agent with all available parameters
agent = Agent(
    role="Senior Data Scientist",
    goal="Analyze and interpret complex datasets to provide actionable insights",
    backstory="With over 10 years of experience in data science and machine learning, "
              "you excel at finding patterns in complex datasets.",
    llm="gpt-4",  # Default: OPENAI_MODEL_NAME or "gpt-4"
    function_calling_llm=None,  # Optional: Separate LLM for tool calling
    verbose=False,  # Default: False
    allow_delegation=False,  # Default: False
    max_iter=20,  # Default: 20 iterations
    max_rpm=None,  # Optional: Rate limit for API calls
    max_execution_time=None,  # Optional: Maximum execution time in seconds
    max_retry_limit=2,  # Default: 2 retries on error
    allow_code_execution=False,  # Default: False
    code_execution_mode="safe",  # Default: "safe" (options: "safe", "unsafe")
    respect_context_window=True,  # Default: True
    use_system_prompt=True,  # Default: True
    multimodal=False,  # Default: False
    inject_date=False,  # Default: False
    date_format="%Y-%m-%d",  # Default: ISO format
    reasoning=False,  # Default: False
    max_reasoning_attempts=None,  # Default: None
    tools=[SerperDevTool()],  # Optional: List of tools
    knowledge_sources=None,  # Optional: List of knowledge sources
    embedder=None,  # Optional: Custom embedder configuration
    system_template=None,  # Optional: Custom system prompt template
    prompt_template=None,  # Optional: Custom prompt template
    response_template=None,  # Optional: Custom response template
    step_callback=None,  # Optional: Callback function for monitoring
)
```

Let's break down some key parameter combinations for common use cases:

Basic Research Agent

```python
research_agent = Agent(
    role="Research Analyst",
    goal="Find and summarize information about specific topics",
    backstory="You are an experienced researcher with attention to detail",
    tools=[SerperDevTool()],
    verbose=True  # Enable logging for debugging
)
```

Code Development Agent

```python
dev_agent = Agent(
    role="Senior Python Developer",
    goal="Write and debug Python code",
    backstory="Expert Python developer with 10 years of experience",
    allow_code_execution=True,
    code_execution_mode="safe",  # Uses Docker for safety
    max_execution_time=300,  # 5-minute timeout
    max_retry_limit=3  # More retries for complex code tasks
)
```

Long-Running Analysis Agent

```
analysis_agent = Agent(
    role="Data Analyst",
    goal="Perform deep analysis of large datasets",
    backstory="Specialized in big data analysis and pattern recognition",
    memory=True,
    respect_context_window=True,
    max_rpm=10,  # Limit API calls
    function_calling_llm="gpt-4o-mini"  # Cheaper model for tool calls
)
```

Custom Template Agent

```
custom_agent = Agent(
    role="Customer Service Representative",
    goal="Assist customers with their inquiries",
    backstory="Experienced in customer support with a focus on satisfaction",
    system_template="""<|start_header_id|>system<|end_header_id|>
                       {{ .System }}<|eot_id|>""",
    prompt_template="""<|start_header_id|>user<|end_header_id|>
                       {{ .Prompt }}<|eot_id|>""",
    response_template="""<|start_header_id|>assistant<|end_header_id|>
                       {{ .Response }}<|eot_id|>""",
)
```

Date-Aware Agent with Reasoning

```
strategic_agent = Agent(
    role="Market Analyst",
    goal="Track market movements with precise date references and strategic planning",
    backstory="Expert in time-sensitive financial analysis and strategic reporting",
    inject_date=True,  # Automatically inject current date into the prompt
    date_format="%B %d, %Y",  # Format as "May 21, 2025"
    reasoning=True,  # Enable strategic planning
    max_reasoning_attempts=2,  # Limit planning iterations
    verbose=True
)
```

Reasoning Agent

```
reasoning_agent = Agent(
    role="Strategic Planner",
    goal="Analyze complex problems and create detailed execution plans",
    backstory="Expert strategic planner who methodically breaks down complex challenges",
    reasoning=True,  # Enable reasoning and planning
    max_reasoning_attempts=3,  # Limit reasoning attempts
    max_iter=30,  # Allow more iterations for complex planning
    verbose=True
)
```

Multimodal Agent

```
multimodal_agent = Agent(
    role="Visual Content Analyst",
    goal="Analyze and process both text and visual content",
    backstory="Specialized in multimodal analysis combining text and image understanding",
    multimodal=True,  # Enable multimodal capabilities
    verbose=True
)
```

Parameter Details

Critical Parameters

- `role`, `goal`, and `backstory` are required and shape the agent's behavior
- `llm` determines the language model used (default: OpenAI's GPT-4)

Memory and Context

- `memory`: Enable to maintain conversation history
- `respect_context_window`: Prevents token limit issues
- `knowledge_sources`: Add domain-specific knowledge bases

Execution Control

- `max_iter`: Maximum attempts before giving best answer
- `max_execution_time`: Timeout in seconds
- `max_rpm`: Rate limiting for API calls
- `max_retry_limit`: Retries on error

Code Execution

> [!WARNING]
>
> `allow_code_execution` and `code_execution_mode` are deprecated. `CodeInterpreterTool` has been removed from `crewai-tools`. Use a dedicated sandbox service such as E2B or Modal for secure code execution.

- `allow_code_execution` *(deprecated)*: Previously enabled built-in code execution via `CodeInterpreterTool`.
- `code_execution_mode` *(deprecated)*: Previously controlled execution mode (`"safe"` for Docker, `"unsafe"` for direct execution).

Advanced Features

- `multimodal`: Enable multimodal capabilities for processing text and visual content
- `reasoning`: Enable agent to reflect and create plans before executing tasks
- `inject_date`: Automatically inject current date into the agents prompt

Templates

- `system_template`: Defines agent's core behavior
- `prompt_template`: Structures input format
- `response_template`: Formats agent responses

> [!NOTE]
>
> When using custom templates, ensure that both `system_template` and `prompt_template` are defined. The `response_template` is optional but recommended for consistent output formatting.

> [!NOTE]
>
> When using custom templates, you can use variables like `{role}`, `{goal}`, and `{backstory}` in your templates. These will be automatically populated during execution.

Agent Tools

Agents can be equipped with various tools to enhance their capabilities. CrewAI supports tools from:

- CrewAI Toolkit
- LangChain Tools

Here's how to add tools to an agent:

```python
from crewai import Agent
from crewai_tools import SerperDevTool, WikipediaTools

# Create tools
search_tool = SerperDevTool()
wiki_tool = WikipediaTools()

# Add tools to agent
researcher = Agent(
    role="AI Technology Researcher",
    goal="Research the latest AI developments",
    tools=[search_tool, wiki_tool],
    verbose=True
)
```

Agent Memory and Context

Agents can maintain memory of their interactions and use context from previous tasks. This is particularly useful for complex workflows where information needs to be retained across multiple tasks.

```python
from crewai import Agent

analyst = Agent(
    role="Data Analyst",
    goal="Analyze and remember complex data patterns",
    memory=True,  # Enable memory
    verbose=True
)
```

> [!NOTE]
>
> When `memory` is enabled, the agent will maintain context across multiple interactions, improving its ability to handle complex, multi-step tasks.

Context Window Management

CrewAI includes sophisticated automatic context window management to handle situations where conversations exceed the language model's token limits. This powerful feature is controlled by the `respect_context_window` parameter.

How Context Window Management Works

When an agent's conversation history grows too large for the LLM's context window, CrewAI automatically detects this situation and can either:

1. **Automatically summarize content** (when `respect_context_window=True`)
2. **Stop execution with an error** (when `respect_context_window=False`)

Automatic Context Handling (`respect_context_window=True`)

This is the **default and recommended setting** for most use cases. When enabled, CrewAI will:

```python
# Agent with automatic context management (default)
smart_agent = Agent(
    role="Research Analyst",
    goal="Analyze large documents and datasets",
    backstory="Expert at processing extensive information",
    respect_context_window=True,  #  Default: auto-handle context limits
    verbose=True
)
```

What happens when context limits are exceeded:

- **Warning message**: `"Context length exceeded. Summarizing content to fit the model context window."`
- **Automatic summarization**: CrewAI intelligently summarizes the conversation history
- **Continued execution**: Task execution continues seamlessly with the summarized context
- **Preserved information**: Key information is retained while reducing token count

Strict Context Limits (`respect_context_window=False`)

When you need precise control and prefer execution to stop rather than lose any information:

```
# Agent with strict context limits
strict_agent = Agent(
    role="Legal Document Reviewer",
    goal="Provide precise legal analysis without information loss",
    backstory="Legal expert requiring complete context for accurate analysis",
    respect_context_window=False,  #  Stop execution on context limit
    verbose=True
)
```

What happens when context limits are exceeded:

- **Error message**: `"Context length exceeded. Consider using smaller text or RAG tools from crewai_tools."`
- **Execution stops**: Task execution halts immediately
- **Manual intervention required**: You need to modify your approach

Choosing the Right Setting

Use `respect_context_window=True` (Default) when:

- **Processing large documents** that might exceed context limits
- **Long-running conversations** where some summarization is acceptable
- **Research tasks** where general context is more important than exact details
- **Prototyping and development** where you want robust execution

```
# Perfect for document processing
document_processor = Agent(
    role="Document Analyst",
    goal="Extract insights from large research papers",
    backstory="Expert at analyzing extensive documentation",
    respect_context_window=True,  # Handle large documents gracefully
    max_iter=50,  # Allow more iterations for complex analysis
    verbose=True
)
```

Use `respect_context_window=False` when:

- **Precision is critical** and information loss is unacceptable
- **Legal or medical tasks** requiring complete context
- **Code review** where missing details could introduce bugs
- **Financial analysis** where accuracy is paramount

```
# Perfect for precision tasks
precision_agent = Agent(
    role="Code Security Auditor",
    goal="Identify security vulnerabilities in code",
    backstory="Security expert requiring complete code context",
    respect_context_window=False,  # Prefer failure over incomplete analysis
    max_retry_limit=1,  # Fail fast on context issues
```

```
    verbose=True
)
```

Alternative Approaches for Large Data

When dealing with very large datasets, consider these strategies:

1. Use RAG Tools

```
from crewai_tools import RagTool

# Create RAG tool for large document processing
rag_tool = RagTool()

rag_agent = Agent(
    role="Research Assistant",
    goal="Query large knowledge bases efficiently",
    backstory="Expert at using RAG tools for information retrieval",
    tools=[rag_tool],  # Use RAG instead of large context windows
    respect_context_window=True,
    verbose=True
)
```

2. Use Knowledge Sources

```
# Use knowledge sources instead of large prompts
knowledge_agent = Agent(
    role="Knowledge Expert",
    goal="Answer questions using curated knowledge",
    backstory="Expert at leveraging structured knowledge sources",
    knowledge_sources=[your_knowledge_sources],  # Pre-processed knowledge
    respect_context_window=True,
    verbose=True
)
```

Context Window Best Practices

1. **Monitor Context Usage**: Enable `verbose=True` to see context management in action
2. **Design for Efficiency**: Structure tasks to minimize context accumulation
3. **Use Appropriate Models**: Choose LLMs with context windows suitable for your tasks
4. **Test Both Settings**: Try both `True` and `False` to see which works better for your use case
5. **Combine with RAG**: Use RAG tools for very large datasets instead of relying solely on context windows

Troubleshooting Context Issues

If you're getting context limit errors:

```
# Quick fix: Enable automatic handling
agent.respect_context_window = True

# Better solution: Use RAG tools for large data
from crewai_tools import RagTool
agent.tools = [RagTool()]

# Alternative: Break tasks into smaller pieces
# Or use knowledge sources instead of large prompts
```

If automatic summarization loses important information:

```python
# Disable auto-summarization and use RAG instead
agent = Agent(
    role="Detailed Analyst",
    goal="Maintain complete information accuracy",
    backstory="Expert requiring full context",
    respect_context_window=False,  # No summarization
    tools=[RagTool()],  # Use RAG for large data
    verbose=True
)
```

[!NOTE]

The context window management feature works automatically in the background. You don't need to call any special functions - just set `respect_context_window` to your preferred behavior and CrewAI handles the rest!

Direct Agent Interaction with `kickoff()`

Agents can be used directly without going through a task or crew workflow using the `kickoff()` method. This provides a simpler way to interact with an agent when you don't need the full crew orchestration capabilities.

How `kickoff()` Works

The `kickoff()` method allows you to send messages directly to an agent and get a response, similar to how you would interact with an LLM but with all the agent's capabilities (tools, reasoning, etc.).

```python
from crewai import Agent
from crewai_tools import SerperDevTool

# Create an agent
researcher = Agent(
    role="AI Technology Researcher",
    goal="Research the latest AI developments",
    tools=[SerperDevTool()],
    verbose=True
)

# Use kickoff() to interact directly with the agent
result = researcher.kickoff("What are the latest developments in language models?")

# Access the raw response
print(result.raw)
```

Parameters and Return Values

Parameter	Type	Description
`messages`	`Union[str, List[Dict[str, str]]]`	Either a string query or a list of message dictionaries with role/content
`response_format`	`Optional[Type[Any]]`	Optional Pydantic model for structured output

The method returns a `LiteAgentOutput` object with the following properties:

- `raw`: String containing the raw output text
- `pydantic`: Parsed Pydantic model (if a `response_format` was provided)
- `agent_role`: Role of the agent that produced the output
- `usage_metrics`: Token usage metrics for the execution

Structured Output

You can get structured output by providing a Pydantic model as the `response_format`:

```
from pydantic import BaseModel
from typing import List

class ResearchFindings(BaseModel):
    main_points: List[str]
    key_technologies: List[str]
    future_predictions: str

# Get structured output
result = researcher.kickoff(
    "Summarize the latest developments in AI for 2025",
    response_format=ResearchFindings
)

# Access structured data
print(result.pydantic.main_points)
print(result.pydantic.future_predictions)
```

Multiple Messages

You can also provide a conversation history as a list of message dictionaries:

```
messages = [
    {"role": "user", "content": "I need information about large language models"},
    {"role": "assistant", "content": "I'd be happy to help with that! What specifically would you like to l
    {"role": "user", "content": "What are the latest developments in 2025?"}
]

result = researcher.kickoff(messages)
```

The last `user` message is the request the agent answers. Every other message keeps its role and its position around that request, so a conversation that ends in assistant or tool messages still asks the user's question and still delivers those trailing turns after it. With no `user` message at all, the last message is treated as the request.

Async Support

An asynchronous version is available via `kickoff_async()` with the same parameters:

```
import asyncio

async def main():
    result = await researcher.kickoff_async("What are the latest developments in AI?")
    print(result.raw)

asyncio.run(main())
```

> [!NOTE]
>
> The `kickoff()` method runs the agent on the same `AgentExecutor` that agents in a crew use, without a task or crew, and keeps all of the agent's configuration (role, goal, backstory, tools, etc.).

Upgrading Agents from Earlier Releases

Release 1.15.29 removed `CrewAgentExecutor` and changed how every agent handles large tool results, long conversations, and repeated tool calls. These changes apply to agents in a crew, in a Flow, and called with `kickoff()`. The exception is the plan steps of an agent that plans: their tool results are not deferred, their conversation is not

summarized, and the repeated-call count restarts for each step. If you are moving from 1.15.28 or earlier, check each one below.

`CrewAgentExecutor` Is Removed

`AgentExecutor` is the only agent executor. It has been the default since 1.14.5, so an agent that never set `executor_class` already ran on it.

- `executor_class=CrewAgentExecutor`, or the string `"CrewAgentExecutor"` in a config file, fails validation with `CrewAgentExecutor has been removed. Use crewai.agents.agent_executor.AgentExecutor.` Remove the setting.
- These import paths are gone: `crewai.agents.crew_agent_executor`, `crewai.agents.CrewAgentExecutor`, the `crewai.experimental.agent_executor` module, `crewai.experimental.AgentExecutor`, and `crewai.experimental.CrewAgentExecutorFlow`. Import the executor from `crewai.agents`:

```
from crewai.agents import AgentExecutor
```

There is no setting that restores the old executor.

Oversized Tool Results Are Deferred

When a native tool call returns a result larger than 25% of the model's context window, the agent no longer receives the whole result. It receives a short preview and a reference to a temporary file, and reads the rest in sections with the `Read a deferred tool result` tool (the model sees it as `read_a_deferred_tool_result`), which CrewAI adds to the agent when it is needed.

- Failed tool calls, tools with `result_as_answer=True`, and text-based (ReAct) tool calls are never deferred.
- The file lives in a private directory under the system temp directory. It can be read only during the run that produced it, and it is removed when that task or `kickoff()` call ends.

To keep full tool results in the conversation, turn deferral off for the agent:

```
from crewai import Agent

agent = Agent(
    role="Source Reviewer",
    goal="Quote sources exactly",
    backstory="Checks every quote against the original text",
    use_better_compaction=False,  # Keep whole tool results in the conversation
)
```

Context Is Summarized Before It Overflows

With `respect_context_window=True` (the default), the agent now measures every request before sending it. When the request is larger than the model's context window, CrewAI summarizes the earlier conversation first, instead of waiting for the provider to reject the request. A summary can lose details such as exact quotes.

Set `respect_context_window=False` to turn summarization off. A request that is too long then stops the run with `SystemExit("Context length exceeded and user opted not to summarize...")`. `SystemExit` is not an `Exception` subclass, so an `except Exception` block does not catch it.

Repeated Tool Calls Stop the Task

When an agent calls the same tool with the same arguments and gets the same result five times in one run, CrewAI stops the task with `ToolExecutionFailedError` and a message that the agent `called '<tool>' with the same arguments 5 times and got the same result each time`.

- The count covers the whole task or `kickoff()` call, not only consecutive calls, and it includes results answered from the tool cache.
- The stop happens whatever `tool_failure_policy` is set, and `max_retry_limit` does not retry it.
- The limit of five is fixed. There is no setting to change or disable it.

How the Context Window Size Is Chosen

Deferral and summarization are both measured against the context window CrewAI uses for the model: 85% of the window listed in its bundled LiteLLM model catalog. A model missing from the catalog falls back to a provider default. For OpenAI, Azure, Bedrock, OpenAI-compatible (such as OpenRouter), and LiteLLM-routed models that default is 8,192 tokens, so CrewAI uses 6,963 after the 85% ratio, and both features start far earlier than the model's real window warrants. A custom `BaseLLM` subclass that does not override `get_context_window_size()` uses 4,096. To see the size CrewAI uses, call `get_context_window_size()` on the LLM. The LLM is built as it would be for a run, so the provider's API key must be set:

```
from crewai import LLM

llm = LLM(model="openrouter/your-provider/your-model")
print(llm.get_context_window_size())
```

Important Considerations and Best Practices

Security and Code Execution

> [!WARNING]
>
> `allow_code_execution` and `code_execution_mode` are deprecated and `CodeInterpreterTool` has been removed. Use a dedicated sandbox service such as E2B or Modal for secure code execution.

Performance Optimization

- Use `respect_context_window: true` to prevent token limit issues
- Set appropriate `max_rpm` to avoid rate limiting
- Enable `cache: true` to improve performance for repetitive tasks
- Adjust `max_iter` and `max_retry_limit` based on task complexity

Memory and Context Management

- Leverage `knowledge_sources` for domain-specific information
- Configure `embedder` when using custom embedding models
- Use custom templates (`system_template`, `prompt_template`, `response_template`) for fine-grained control over agent behavior

Advanced Features

- Enable `reasoning: true` for agents that need to plan and reflect before executing complex tasks
- Set appropriate `max_reasoning_attempts` to control planning iterations (None for unlimited attempts)
- Use `inject_date: true` to provide agents with current date awareness for time-sensitive tasks
- Customize the date format with `date_format` using standard Python datetime format codes
- Enable `multimodal: true` for agents that need to process both text and visual content

Agent Collaboration

- Enable `allow_delegation: true` when agents need to work together
- Use `step_callback` to monitor and log agent interactions
- Consider using different LLMs for different purposes:
  - Main `llm` for complex reasoning
  - `function_calling_llm` for efficient tool usage

Date Awareness and Reasoning

- Use `inject_date: true` to provide agents with current date awareness for time-sensitive tasks
- Customize the date format with `date_format` using standard Python datetime format codes
- Valid format codes include: %Y (year), %m (month), %d (day), %B (full month name), etc.
- Invalid date formats will be logged as warnings and will not modify the task description

- Enable `reasoning: true` for complex tasks that benefit from upfront planning and reflection

Model Compatibility

- Set `use_system_prompt: false` for older models that don't support system messages
- Ensure your chosen `llm` supports the features you need (like function calling)

Troubleshooting Common Issues

1. **Rate Limiting**: If you're hitting API rate limits:
   - Implement appropriate `max_rpm`
   - Use caching for repetitive operations
   - Consider batching requests
2. **Context Window Errors**: If you're exceeding context limits:
   - Enable `respect_context_window`
   - Use more efficient prompts
   - Clear agent memory periodically
3. **Code Execution Issues**: If code execution fails:
   - Verify Docker is installed for safe mode
   - Check execution permissions
   - Review code sandbox settings
4. **Memory Issues**: If agent responses seem inconsistent:
   - Check knowledge source configuration
   - Review conversation history management

Remember that agents are most effective when configured according to their specific use case. Take time to understand your requirements and adjust these parameters accordingly.

This documentation is built and hosted on Mintlify, a developer documentation platform.

Crews

Understanding and utilizing crews in the crewAI framework with comprehensive attributes and functionalities.

Overview

A crew in crewAI represents a collaborative group of agents working together to achieve a set of tasks. Each crew defines the strategy for task execution, agent collaboration, and the overall workflow.

Crew Attributes

Attribute	Parameters	Description
Tasks	`tasks`	A list of tasks assigned to the crew.
Agents	`agents`	A list of agents that are part of the crew.
Process *(optional)*	`process`	The process flow (e.g., sequential, hierarchical) the crew follows. Default is `sequential`.
Verbose *(optional)*	`verbose`	The verbosity level for logging during execution. Defaults to `False`.
Manager LLM *(optional)*	`manager_llm`	The language model used by the manager agent in a hierarchical process. **Required when using a hierarchical process.**
Function Calling LLM *(optional)*	`function_calling_llm`	If passed, the crew will use this LLM to do function calling for tools for all agents in the crew. Each agent can have its own LLM, which overrides the crew's LLM for function calling.
Config *(optional)*	`config`	Optional configuration settings for the crew, in `Json` or `Dict[str, Any]` format.
Max RPM *(optional)*	`max_rpm`	Maximum requests per minute the crew adheres to during execution. Defaults to `None`.
Memory *(optional)*	`memory`	Utilized for storing execution memories (short-term, long-term, entity memory).
Cache *(optional)*	`cache`	Specifies whether to use a cache for storing the results of tools' execution. Defaults to `True`.
Embedder *(optional)*	`embedder`	Configuration for the embedder to be used by the crew. Mostly used by memory for now. Default is `{"provider": "openai"}`.

Attribute	Parameters	Description
Step Callback *(optional)*	`step_callback`	A function that is called after each step of every agent. This can be used to log the agent's actions or to perform other operations; it won't override the agent-specific `step_callback`.
Task Callback *(optional)*	`task_callback`	A function that is called after the completion of each task. Useful for monitoring or additional operations post-task execution.
Share Crew *(optional)*	`share_crew`	Whether you want to share the complete crew information and execution with the crewAI team to make the library better, and allow us to train models.
Output Log File *(optional)*	`output_log_file`	Set to True to save logs as logs.txt in the current directory or provide a file path. Logs will be in JSON format if the filename ends in .json, otherwise .txt. Defaults to `None`.
Manager Agent *(optional)*	`manager_agent`	`manager` sets a custom agent that will be used as a manager.
Prompt File *(optional)*	`prompt_file`	Path to the prompt JSON file to be used for the crew.
Planning *(optional)*	`planning`	Adds planning ability to the Crew. When activated before each Crew iteration, all Crew data is sent to an AgentPlanner that will plan the tasks and this plan will be added to each task description.
Planning LLM *(optional)*	`planning_llm`	The language model used by the AgentPlanner in a planning process.
Knowledge Sources *(optional)*	`knowledge_sources`	Knowledge sources available at the crew level, accessible to all the agents.
Stream *(optional)*	`stream`	Enable streaming output to receive real-time updates during crew execution. Returns a `CrewStreamingOutput` object that can be iterated for chunks. Defaults to `False`.

Attribute	Parameters	Description
Chat LLM *(optional)*	`chat_llm`	The language model used to orchestrate `crewai chat` CLI interactions with the crew. Accepts a model name string or `LLM` instance. Defaults to `None`.
Before Kickoff Callbacks *(optional)*	`before_kickoff_callbacks`	A list of callable functions executed **before** the crew starts. Each callback receives and can modify the inputs dict. Distinct from the `@before_kickoff` decorator. Defaults to `[]`.
After Kickoff Callbacks *(optional)*	`after_kickoff_callbacks`	A list of callable functions executed **after** the crew finishes. Each callback receives and can modify the `CrewOutput`. Distinct from the `@after_kickoff` decorator. Defaults to `[]`.
Tracing *(optional)*	`tracing`	Controls tracing for the crew. `True` = always enable, `False` = always disable, `None` = inherit from environment / user settings. Defaults to `None`.
Skills *(optional)*	`skills`	A list of `Path` objects (skill search directories) or pre-loaded `Skill` objects applied to all agents in the crew. Defaults to `None`.
Security Config *(optional)*	`security_config`	A `SecurityConfig` instance managing crew fingerprinting and identity. Defaults to `SecurityConfig()`.
Checkpoint *(optional)*	`checkpoint`	Enables automatic checkpointing. Pass `True` for sensible defaults, a `CheckpointConfig` for full control, `False` to opt out, or `None` to inherit. See the Checkpointing section below. Defaults to `None`.

[!TIP]

Crew Max RPM: The `max_rpm` attribute sets the maximum number of requests per minute the crew can perform to avoid rate limits and will override individual agents' `max_rpm` settings if you set it.

Creating Crews

There are two common ways to create crews in CrewAI: using **JSONC project configuration (recommended for new crews)** or defining them **directly in code**.

JSONC Configuration (Recommended)

New projects created with `crewai create crew <name>` use `crew.jsonc` for crew-level settings and tasks, plus one file per agent in `agents/`.

`crewai run` automatically detects `crew.jsonc` or `crew.json`, loads the referenced agents, prompts for missing placeholders, and kicks off the crew.

Example `crew.jsonc`

```
{
  "name": "Market Research Crew",
  "agents": ["researcher", "analyst"],
  "tasks": [
    {
      "name": "research",
      "description": "Research {topic} and collect the most relevant facts.",
      "expected_output": "Structured research notes about {topic}.",
      "agent": "researcher"
    },
    {
      "name": "analysis",
      "description": "Analyze the research and write a concise report.",
      "expected_output": "A markdown report with findings and recommendations.",
      "agent": "analyst",
      "context": ["research"],
      "output_file": "output/report.md"
    }
  ],
  "process": "sequential",
  "verbose": true,
  "memory": true,
  "inputs": {
    "topic": "AI Agents"
  }
}
```

Each string in `agents` resolves to `agents/<name>.jsonc` first, then `agents/<name>.json`.

```
{
  "role": "{topic} Senior Researcher",
  "goal": "Find accurate and current information about {topic}.",
  "backstory": "You are a careful researcher who cites clear evidence.",
  "llm": "openai/gpt-4o",
  "tools": ["SerperDevTool"]
}
```

> [!NOTE]
>
> Tasks run in the order they appear in `tasks` when `process` is `"sequential"`.

For hierarchical crews, set `"process": "hierarchical"` and provide either `manager_llm` or `manager_agent`. A `manager_agent` can reference an `agents/<name>.jsonc` file that is not included in the top-level `agents` list.

JSON crew definitions support crew-level fields such as `process`, `verbose`, `memory`, `cache`, `max_rpm`, `planning`, `planning_llm`, `manager_llm`, `manager_agent`, `function_calling_llm`, `output_log_file`, `stream`, `tracing`, `before_kickoff_callbacks`, and `after_kickoff_callbacks`.

Python callbacks and custom classes use `{"python": "module.attribute"}`. Custom tools use `"custom:<name>"` and load `tools/<name>.py` at runtime.

> [!WARNING]

Only run JSON crew projects from sources you trust. `custom:<name>` tools and `{"python": "module.attribute"}` references execute local Python code when the crew loads.

Classic Python/YAML Configuration

Classic projects created with `crewai create crew <name> --classic` use `crew.py`, `config/agents.yaml`, `config/tasks.yaml`, and the `@CrewBase`, `@agent`, `@task`, and `@crew` decorators. That pattern remains supported and is documented in Using Annotations.

Direct Code Definition (Alternative)

Alternatively, you can define the crew directly in code without using YAML configuration files.

```
from crewai import Agent, Crew, Task, Process
from crewai_tools import YourCustomTool

class YourCrewName:
    def agent_one(self) -> Agent:
        return Agent(
            role="Data Analyst",
            goal="Analyze data trends in the market",
            backstory="An experienced data analyst with a background in economics",
            verbose=True,
            tools=[YourCustomTool()]
        )

    def agent_two(self) -> Agent:
        return Agent(
            role="Market Researcher",
            goal="Gather information on market dynamics",
            backstory="A diligent researcher with a keen eye for detail",
            verbose=True
        )

    def task_one(self) -> Task:
        return Task(
            description="Collect recent market data and identify trends.",
            expected_output="A report summarizing key trends in the market.",
            agent=self.agent_one()
        )

    def task_two(self) -> Task:
        return Task(
            description="Research factors affecting market dynamics.",
            expected_output="An analysis of factors influencing the market.",
            agent=self.agent_two()
        )

    def crew(self) -> Crew:
        return Crew(
            agents=[self.agent_one(), self.agent_two()],
            tasks=[self.task_one(), self.task_two()],
            process=Process.sequential,
            verbose=True
        )
```

How to run the above code:

```
YourCrewName().crew().kickoff(inputs={})
```

In this example:

- Agents and tasks are defined directly within the class without decorators.
- We manually create and manage the list of agents and tasks.
- This approach provides more control but can be less maintainable for larger projects.

Crew Output

The output of a crew in the CrewAI framework is encapsulated within the `CrewOutput` class. This class provides a structured way to access results of the crew's execution, including various formats such as raw strings, JSON, and Pydantic models. The `CrewOutput` includes the results from the final task output, token usage, and individual task outputs.

Crew Output Attributes

Attribute	Parameters	Type	Description
Raw	`raw`	`str`	The raw output of the crew. This is the default format for the output.
Pydantic	`pydantic`	`Optional[BaseModel]`	A Pydantic model object representing the structured output of the crew.
JSON Dict	`json_dict`	`Optional[Dict[str, Any]]`	A dictionary representing the JSON output of the crew.
Tasks Output	`tasks_output`	`List[TaskOutput]`	A list of `TaskOutput` objects, each representing the output of a task in the crew.
Token Usage	`token_usage`	`Dict[str, Any]`	A summary of token usage, providing insights into the language model's performance during execution.

Crew Output Methods and Properties

Method/Property	Description
json	Returns the JSON string representation of the crew output if the output format is JSON, or `None` if the output could not be converted.
to_dict	Converts the JSON and Pydantic outputs to a dictionary.
***str**	Returns the string representation of the crew output, prioritizing Pydantic, then JSON, then raw.

Accessing Crew Outputs

Once a crew has been executed, its output can be accessed through the `output` attribute of the `Crew` object. The `CrewOutput` class provides various ways to interact with and present this output.

Example

```
# Example crew execution
crew = Crew(
```

```
    agents=[research_agent, writer_agent],
    tasks=[research_task, write_article_task],
    verbose=True
)

crew_output = crew.kickoff()

# Accessing the crew output
print(f"Raw Output: {crew_output.raw}")
if crew_output.json_dict:
    print(f"JSON Output: {json.dumps(crew_output.json_dict, indent=2)}")
if crew_output.pydantic:
    print(f"Pydantic Output: {crew_output.pydantic}")
print(f"Tasks Output: {crew_output.tasks_output}")
print(f"Token Usage: {crew_output.token_usage}")
```

Accessing Crew Logs

You can see real time log of the crew execution, by setting `output_log_file` as a `True(Boolean)` or a `file_name(str)`. Supports logging of events as both `file_name.txt` and `file_name.json`. In case of `True(Boolean)` will save as `logs.txt`.

In case of `output_log_file` is set as `False(Boolean)` or `None`, the logs will not be populated.

```
# Save crew logs
crew = Crew(output_log_file = True)  # Logs will be saved as logs.txt
crew = Crew(output_log_file = file_name)  # Logs will be saved as file_name.txt
crew = Crew(output_log_file = file_name.txt)  # Logs will be saved as file_name.txt
crew = Crew(output_log_file = file_name.json)  # Logs will be saved as file_name.json
```

Checkpointing

Checkpointing lets a crew automatically save its state after key events (e.g. task completion) so that long-running or interrupted runs can be resumed exactly where they left off without re-executing completed tasks.

Quick Start

Pass `checkpoint=True` to enable checkpointing with sensible defaults (saves to `.checkpoints/` after every task):

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential,
    checkpoint=True,  # saves to .checkpoints/ after every task
)

crew.kickoff(inputs={"topic": "AI trends"})
```

Full Control with `CheckpointConfig`

Use `CheckpointConfig` for fine-grained control over location, trigger events, storage backend, and retention:

```
from crewai import Crew, Process
from crewai.state.checkpoint_config import CheckpointConfig

crew = Crew(
    agents=[researcher, writer],
```

```
    tasks=[research_task, write_task],
    process=Process.sequential,
    checkpoint=CheckpointConfig(
        location="./.checkpoints",       # directory for JSON files (default)
        on_events=["task_completed"],    # trigger after each task (default)
        max_checkpoints=5,               # keep only the 5 most recent checkpoints
    ),
)

crew.kickoff(inputs={"topic": "AI trends"})
```

Resuming from a Checkpoint

Use `Crew.from_checkpoint()` to restore a crew from a saved checkpoint file, then call `kickoff()` to resume:

```
# Resume from the most recent checkpoint
crew = Crew.from_checkpoint(".checkpoints/latest.json")
crew.kickoff()
```

> [!NOTE]
>
> When restoring from a checkpoint, `checkpoint_inputs`, `checkpoint_train`, and `checkpoint_kickoff_event_id` are automatically reconstructed – you do not need to set these manually.

`CheckpointConfig` Attributes

Attribute	Type	Default	Description
`location`	`str`	`"./.checkpoints"`	Storage destination. For `JsonProvider` this is a directory path; for `SqliteProvider` a database file path.
`on_events`	`list[str]`	`["task_completed"]`	Event types that trigger a checkpoint write. Use `["*"]` to checkpoint on every event.
`provider`	`JsonProvider | SqliteProvider`	`JsonProvider()`	Storage backend. Defaults to `JsonProvider` (plain JSON files).
`max_checkpoints`	`int | None`	`None`	Maximum checkpoints to keep. Oldest are pruned after each write. `None` keeps all.

Memory Utilization

Crews can utilize memory (short-term, long-term, and entity memory) to enhance their execution and learning over time. This feature allows crews to store and recall execution memories, aiding in decision-making and task execution strategies.

Cache Utilization

Caches can be employed to store the results of tools' execution, making the process more efficient by reducing the need to re-execute identical tasks.

Crew Usage Metrics

After the crew execution, you can access the `usage_metrics` attribute to view the language model (LLM) usage metrics for all tasks executed by the crew. This provides insights into operational efficiency and areas for improvement.

`total_tokens` is the billed total (`prompt_tokens + completion_tokens`). Breakdown fields such as `cached_prompt_tokens` and `cache_creation_tokens` describe subsets already included in those totals and are not added on top of `total_tokens`. See the **UsageMetrics field semantics** section in the Flows concept documentation for the full contract.

```
# Access the crew's usage metrics
crew = Crew(agents=[agent1, agent2], tasks=[task1, task2])
crew.kickoff()
print(crew.usage_metrics)
```

Crew Execution Process

- **Sequential Process**: Tasks are executed one after another, allowing for a linear flow of work.
- **Hierarchical Process**: A manager agent coordinates the crew, delegating tasks and validating outcomes before proceeding. **Note**: A `manager_llm` or `manager_agent` is required for this process and it's essential for validating the process flow.

Kicking Off a Crew

Once your crew is assembled, initiate the workflow with the `kickoff()` method. This starts the execution process according to the defined process flow.

```
# Start the crew's task execution
result = my_crew.kickoff()
print(result)
```

Different Ways to Kick Off a Crew

Once your crew is assembled, initiate the workflow with the appropriate kickoff method. CrewAI provides several methods for better control over the kickoff process.

Synchronous Methods

- `kickoff()`: Starts the execution process according to the defined process flow.
- `kickoff_for_each()`: Executes tasks sequentially for each provided input event or item in the collection.

Asynchronous Methods CrewAI offers two approaches for async execution:

Method	Type	Description
`akickoff()`	Native async	True async/await throughout the entire execution chain
`akickoff_for_each()`	Native async	Native async execution for each input in a list
`kickoff_async()`	Thread-based	Wraps synchronous execution in `asyncio.to_thread`
`kickoff_for_each_async()`	Thread-based	Thread-based async for each input in a list

[!NOTE]

For high-concurrency workloads, `akickoff()` and `akickoff_for_each()` are recommended as they use native async for task execution, memory operations, and knowledge retrieval.

```python
# Start the crew's task execution
result = my_crew.kickoff()
print(result)

# Example of using kickoff_for_each
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
results = my_crew.kickoff_for_each(inputs=inputs_array)
for result in results:
    print(result)

# Example of using native async with akickoff
inputs = {'topic': 'AI in healthcare'}
async_result = await my_crew.akickoff(inputs=inputs)
print(async_result)

# Example of using native async with akickoff_for_each
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
async_results = await my_crew.akickoff_for_each(inputs=inputs_array)
for async_result in async_results:
    print(async_result)

# Example of using thread-based kickoff_async
inputs = {'topic': 'AI in healthcare'}
async_result = await my_crew.kickoff_async(inputs=inputs)
print(async_result)

# Example of using thread-based kickoff_for_each_async
inputs_array = [{'topic': 'AI in healthcare'}, {'topic': 'AI in finance'}]
async_results = await my_crew.kickoff_for_each_async(inputs=inputs_array)
for async_result in async_results:
    print(async_result)
```

These methods provide flexibility in how you manage and execute tasks within your crew, allowing for both synchronous and asynchronous workflows tailored to your needs. For detailed async examples, see the Kickoff Crew Asynchronously guide.

Streaming Crew Execution

For real-time visibility into crew execution, you can enable streaming to receive output as it's generated:

```python
# Enable streaming
crew = Crew(
    agents=[researcher],
    tasks=[task],
    stream=True
)

# Iterate over streaming output
streaming = crew.kickoff(inputs={"topic": "AI"})
for chunk in streaming:
    print(chunk.content, end="", flush=True)

# Access final result
result = streaming.result
```

Learn more about streaming in the Streaming Crew Execution guide.

Replaying from a Specific Task

You can now replay from a specific task using our CLI command `replay`.

The replay feature in CrewAI allows you to replay from a specific task using the command-line interface (CLI). By running the command `crewai replay -t <task_id>`, you can specify the `task_id` for the replay process.

Kickoffs will now save the latest kickoffs returned task outputs locally for you to be able to replay from.

Replaying from a Specific Task Using the CLI

To use the replay feature, follow these steps:

1. Open your terminal or command prompt.
2. Navigate to the directory where your CrewAI project is located.
3. Run the following command:

To view the latest kickoff task IDs, use:

```
crewai log-tasks-outputs
```

Then, to replay from a specific task, use:

```
crewai replay -t <task_id>
```

These commands let you replay from your latest kickoff tasks, still retaining context from previously executed tasks.

This documentation is built and hosted on Mintlify, a developer documentation platform.

Flows

Learn how to create and manage AI workflows using CrewAI Flows.

Overview

CrewAI Flows is a powerful feature designed to streamline the creation and management of AI workflows. Flows allow developers to combine and coordinate coding tasks and Crews efficiently, providing a robust framework for building sophisticated AI automations.

Flows allow you to create structured, event-driven workflows. They provide a seamless way to connect multiple tasks, manage state, and control the flow of execution in your AI applications. With Flows, you can easily design and implement multi-step processes that leverage the full potential of CrewAI's capabilities.

1. **Simplified Workflow Creation**: Easily chain together multiple Crews and tasks to create complex AI workflows.
2. **State Management**: Flows make it super easy to manage and share state between different tasks in your workflow.
3. **Event-Driven Architecture**: Built on an event-driven model, allowing for dynamic and responsive workflows.
4. **Flexible Control Flow**: Implement conditional logic, loops, and branching within your workflows.

Getting Started

Let's create a simple Flow where you will use OpenAI to generate a random city in one task and then use that city to generate a fun fact in another task.

```
from crewai.flow.flow import Flow, listen, start
from dotenv import load_dotenv
from litellm import completion

load_dotenv()

class ExampleFlow(Flow):
    model = "gpt-4o-mini"

    @start()
    def generate_city(self):
        print("Starting flow")
        # Each flow state automatically gets a unique ID
        print(f"Flow State ID: {self.state['id']}")

        response = completion(
            model=self.model,
            messages=[
                {
                    "role": "user",
                    "content": "Return the name of a random city in the world.",
                },
            ],
        )

        random_city = response["choices"][0]["message"]["content"]
        # Store the city in our state
        self.state["city"] = random_city
        print(f"Random City: {random_city}")
```

```
        return random_city

    @listen(generate_city)
    def generate_fun_fact(self, random_city):
        response = completion(
            model=self.model,
            messages=[
                {
                    "role": "user",
                    "content": f"Tell me a fun fact about {random_city}",
                },
            ],
        )

        fun_fact = response["choices"][0]["message"]["content"]
        # Store the fun fact in our state
        self.state["fun_fact"] = fun_fact
        return fun_fact


flow = ExampleFlow()
flow.plot()
result = flow.kickoff()

print(f"Generated fun fact: {result}")
```

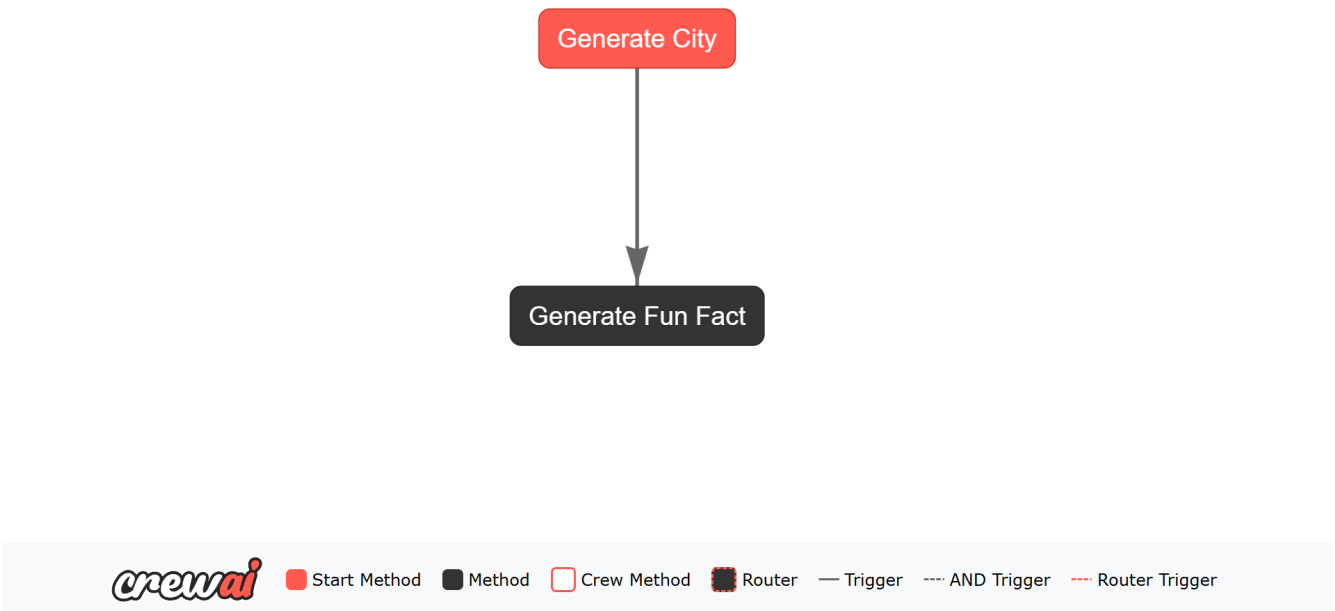


Figure 4: Flow Visual image

In the above example, we have created a simple Flow that generates a random city using OpenAI and then generates a fun fact about that city. The Flow consists of two tasks: `generate_city` and `generate_fun_fact`. The `generate_city` task is the starting point of the Flow, and the `generate_fun_fact` task listens for the output of the `generate_city` task.

Each Flow instance automatically receives a unique identifier (UUID) in its state, which helps track and manage flow

executions. The state can also store additional data (like the generated city and fun fact) that persists throughout the flow's execution.

When you run the Flow, it will:

1. Generate a unique ID for the flow state
2. Generate a random city and store it in the state
3. Generate a fun fact about that city and store it in the state
4. Print the results to the console

The state's unique ID and stored data can be useful for tracking flow executions and maintaining context between tasks.

Note: Ensure you have set up your `.env` file to store your `OPENAI_API_KEY`. This key is necessary for authenticating requests to the OpenAI API.

@start()

The `@start()` decorator marks entry points for a Flow. You can:

- Declare multiple unconditional starts: `@start()`
- Gate a start on a prior method or router label: `@start("method_or_label")`
- Provide a callable condition to control when a start should fire

All satisfied `@start()` methods will execute (often in parallel) when the Flow begins or resumes.

@listen()

The `@listen()` decorator is used to mark a method as a listener for the output of another task in the Flow. The method decorated with `@listen()` will be executed when the specified task emits an output. The method can access the output of the task it is listening to as an argument.

Usage The `@listen()` decorator can be used in several ways:

1. **Listening to a Method by Name**: You can pass the name of the method you want to listen to as a string. When that method completes, the listener method will be triggered.

```
@listen("generate_city")
def generate_fun_fact(self, random_city):
    # Implementation
```

2. **Listening to a Method Directly**: You can pass the method itself. When that method completes, the listener method will be triggered.

```
@listen(generate_city)
def generate_fun_fact(self, random_city):
    # Implementation
```

Flow Output

Accessing and handling the output of a Flow is essential for integrating your AI workflows into larger applications or systems. CrewAI Flows provide straightforward mechanisms to retrieve the final output, access intermediate results, and manage the overall state of your Flow.

Retrieving the Final Output When you run a Flow, the final output is determined by the last method that completes. The `kickoff()` method returns the output of this final method.

Here's how you can access the final output:

```
from crewai.flow.flow import Flow, listen, start

class OutputExampleFlow(Flow):
    @start()
```

```
    def first_method(self):
        return "Output from first_method"

    @listen(first_method)
    def second_method(self, first_output):
        return f"Second method received: {first_output}"


flow = OutputExampleFlow()
flow.plot("my_flow_plot")
final_output = flow.kickoff()

print("---- Final Output ----")
print(final_output)
```

```
---- Final Output ----
Second method received: Output from first_method
```

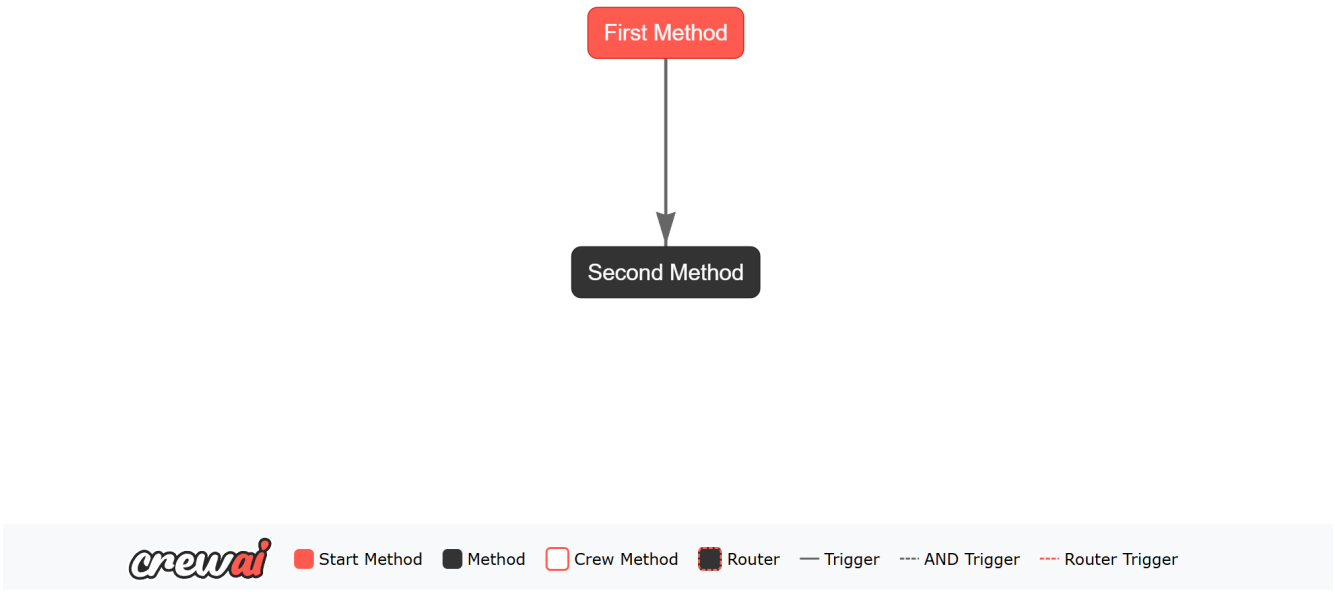


Figure 5: Flow Visual image

In this example, the `second_method` is the last method to complete, so its output will be the final output of the Flow. The `kickoff()` method will return the final output, which is then printed to the console. The `plot()` method will generate the HTML file, which will help you understand the flow.

Accessing and Updating State In addition to retrieving the final output, you can also access and update the state within your Flow. The state can be used to store and share data between different methods in the Flow. After the Flow has run, you can access the state to retrieve any information that was added or updated during the execution.

Here's an example of how to update and access the state:

```
from crewai.flow.flow import Flow, listen, start
from pydantic import BaseModel

class ExampleState(BaseModel):
```

```
    counter: int = 0
    message: str = ""

class StateExampleFlow(Flow[ExampleState]):

    @start()
    def first_method(self):
        self.state.message = "Hello from first_method"
        self.state.counter += 1

    @listen(first_method)
    def second_method(self):
        self.state.message += " - updated by second_method"
        self.state.counter += 1
        return self.state.message

flow = StateExampleFlow()
flow.plot("my_flow_plot")
final_output = flow.kickoff()
print(f"Final Output: {final_output}")
print("Final State:")
print(flow.state)
```

```
Final Output: Hello from first_method - updated by second_method
Final State:
counter=2 message='Hello from first_method - updated by second_method'
```

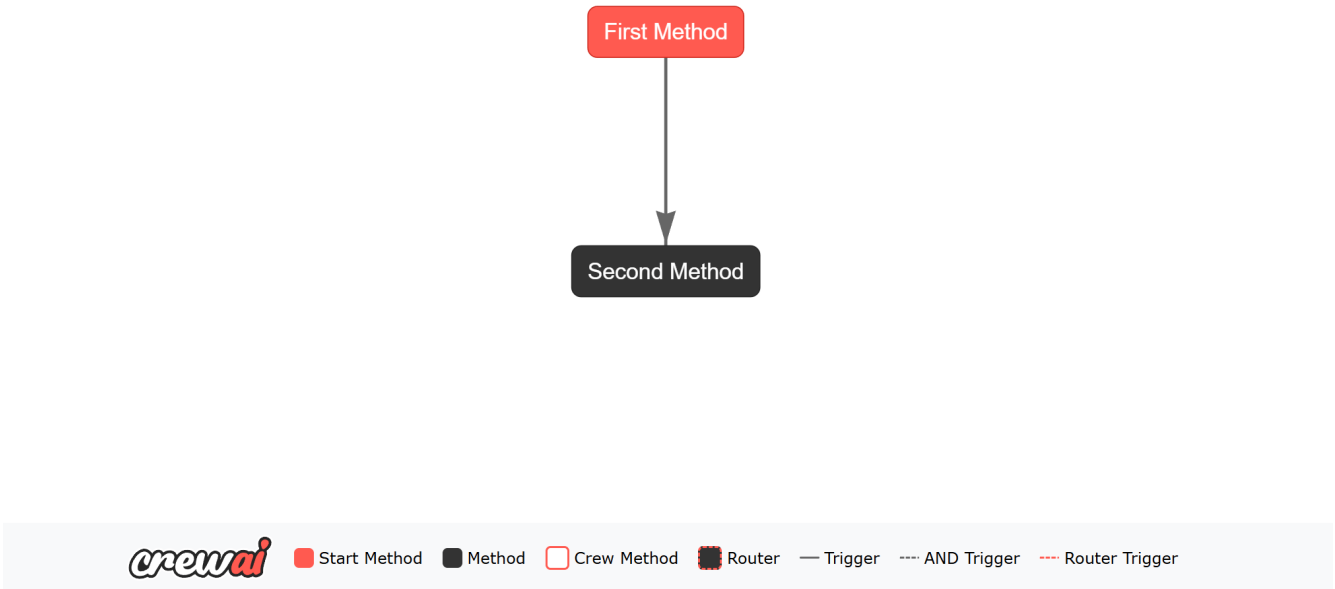


Figure 6: Flow Visual image

In this example, the state is updated by both `first_method` and `second_method`. After the Flow has run, you can access the final state to see the updates made by these methods.

By ensuring that the final method's output is returned and providing access to the state, CrewAI Flows make it easy to integrate the results of your AI workflows into larger applications or systems, while also maintaining and accessing the state throughout the Flow's execution.

Flow Usage Metrics

After a Flow execution completes, you can access the `usage_metrics` property to view aggregated token usage across **every LLM call** made during the run – including calls from every Crew the Flow orchestrated, calls inside Agent tools, and bare `LLM.call(...)` invocations from Flow methods. This is the SDK-side equivalent of the totals shown in the CrewAI Enterprise UI.

```
from crewai import LLM
from crewai.flow.flow import Flow, listen, start

class UsageMetricsFlow(Flow):
    @start()
    def run_first_crew(self):
        self.state.first_result = FirstCrew().crew().kickoff()

    @listen(run_first_crew)
    def call_llm_directly(self):
        # Bare LLM call -- still counted by flow.usage_metrics
        llm = LLM(model="openai/gpt-4o-mini")
        self.state.summary = llm.call("Summarize the key takeaways.")

    @listen(call_llm_directly)
    def run_second_crew(self):
        self.state.second_result = SecondCrew().crew().kickoff()

flow = UsageMetricsFlow()
flow.kickoff()

print(flow.usage_metrics)
# UsageMetrics(total_tokens=8579, prompt_tokens=6210, completion_tokens=2369,
#              cached_prompt_tokens=0, reasoning_tokens=0,
#              cache_creation_tokens=0, successful_requests=5)
```

[!NOTE]

`flow.usage_metrics` is **not** the same as `flow.kickoff().token_usage`. The latter returns the `CrewOutput.token_usage` of the **last** `@listen` method that returned a `CrewOutput`, which means it only reflects the final Crew and ignores prior Crews and bare `LLM.call(...)` invocations entirely. Use `flow.usage_metrics` whenever you need the **full** token rollup for the Flow execution.

UsageMetrics field semantics

The returned `UsageMetrics` object uses a provider-neutral contract:

Field	Meaning
`total_tokens`	Billed total: `prompt_tokens + completion_tokens`
`prompt_tokens`	Full input/prompt tokens billed for the request
`completion_tokens`	Output/completion tokens billed for the request
`cached_prompt_tokens`	Cache-read subset of prompt tokens (breakdown only)
`cache_creation_tokens`	Cache-write subset of prompt tokens (breakdown only, Anthropic)
`reasoning_tokens`	Reasoning/thinking subset where the provider reports it separately (breakdown only)
`successful_requests`	Number of LLM calls aggregated

Breakdown fields such as `cached_prompt_tokens`, `cache_creation_tokens`, and `reasoning_tokens` are **not** added on top of `total_tokens` – they describe portions already included in `prompt_tokens` or `completion_tokens`.

For Anthropic, cache read and cache write counters are folded into `prompt_tokens`, so cached workloads are fully reflected in `total_tokens`. OpenAI-style providers already include cached input inside `prompt_tokens`; CrewAI surfaces the cached portion separately for visibility.

Each entry in the returned `UsageMetrics` is the sum across all LLM calls made within a single `flow.kickoff()` invocation. Counters reset on the next `kickoff()` call (or on each iteration of `kickoff_for_each`), so successive runs don't double-count. The property is safe to read at any point after `kickoff()` completes; reading it during execution returns the partial total accumulated so far.

Flow State Management

Managing state effectively is crucial for building reliable and maintainable AI workflows. CrewAI Flows provides robust mechanisms for both unstructured and structured state management, allowing developers to choose the approach that best fits their application's needs.

Unstructured State Management

In unstructured state management, all state is stored in the `state` attribute of the `Flow` class. This approach offers flexibility, enabling developers to add or modify state attributes on the fly without defining a strict schema. Even with unstructured states, CrewAI Flows automatically generates and maintains a unique identifier (UUID) for each state instance.

```
from crewai.flow.flow import Flow, listen, start

class UnstructuredExampleFlow(Flow):

    @start()
    def first_method(self):
        # The state automatically includes an 'id' field
        print(f"State ID: {self.state['id']}")
        self.state['counter'] = 0
        self.state['message'] = "Hello from structured flow"

    @listen(first_method)
    def second_method(self):
        self.state['counter'] += 1
        self.state['message'] += " - updated"

    @listen(second_method)
    def third_method(self):
        self.state['counter'] += 1
        self.state['message'] += " - updated again"

        print(f"State after third_method: {self.state}")


flow = UnstructuredExampleFlow()
flow.plot("my_flow_plot")
flow.kickoff()
```

Note: The `id` field is automatically generated and preserved throughout the flow's execution. You don't need to manage or set it manually, and it will be maintained even when updating the state with new data.

Key Points:

- **Flexibility:** You can dynamically add attributes to `self.state` without predefined constraints.
- **Simplicity:** Ideal for straightforward workflows where state structure is minimal or varies significantly.

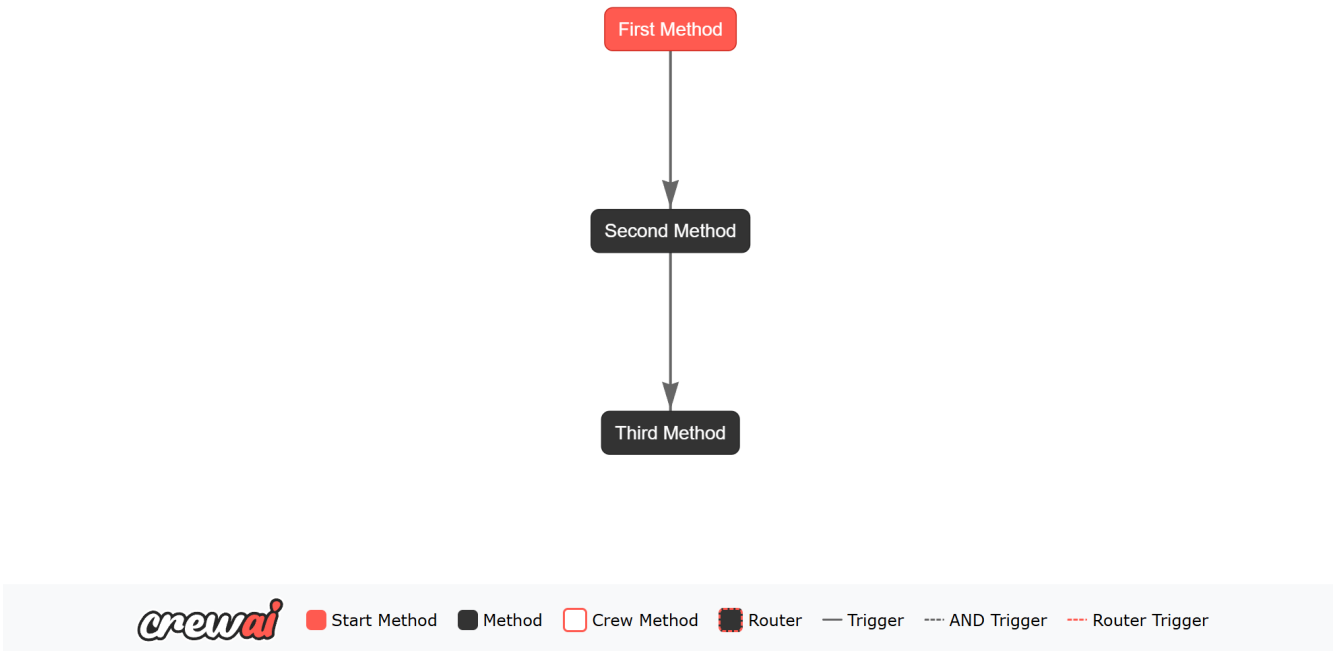


Figure 7: Flow Visual image

Structured State Management

Structured state management leverages predefined schemas to ensure consistency and type safety across the workflow. By using models like Pydantic's `BaseModel`, developers can define the exact shape of the state, enabling better validation and auto-completion in development environments.

Each state in CrewAI Flows automatically receives a unique identifier (UUID) to help track and manage state instances. This ID is automatically generated and managed by the Flow system.

```
from crewai.flow.flow import Flow, listen, start
from pydantic import BaseModel


class ExampleState(BaseModel):
    # Note: 'id' field is automatically added to all states
    counter: int = 0
    message: str = ""


class StructuredExampleFlow(Flow[ExampleState]):

    @start()
    def first_method(self):
        # Access the auto-generated ID if needed
        print(f"State ID: {self.state.id}")
        self.state.message = "Hello from structured flow"

    @listen(first_method)
    def second_method(self):
        self.state.counter += 1
        self.state.message += " - updated"
```

```
    @listen(second_method)
    def third_method(self):
        self.state.counter += 1
        self.state.message += " - updated again"

        print(f"State after third_method: {self.state}")


flow = StructuredExampleFlow()
flow.kickoff()
```

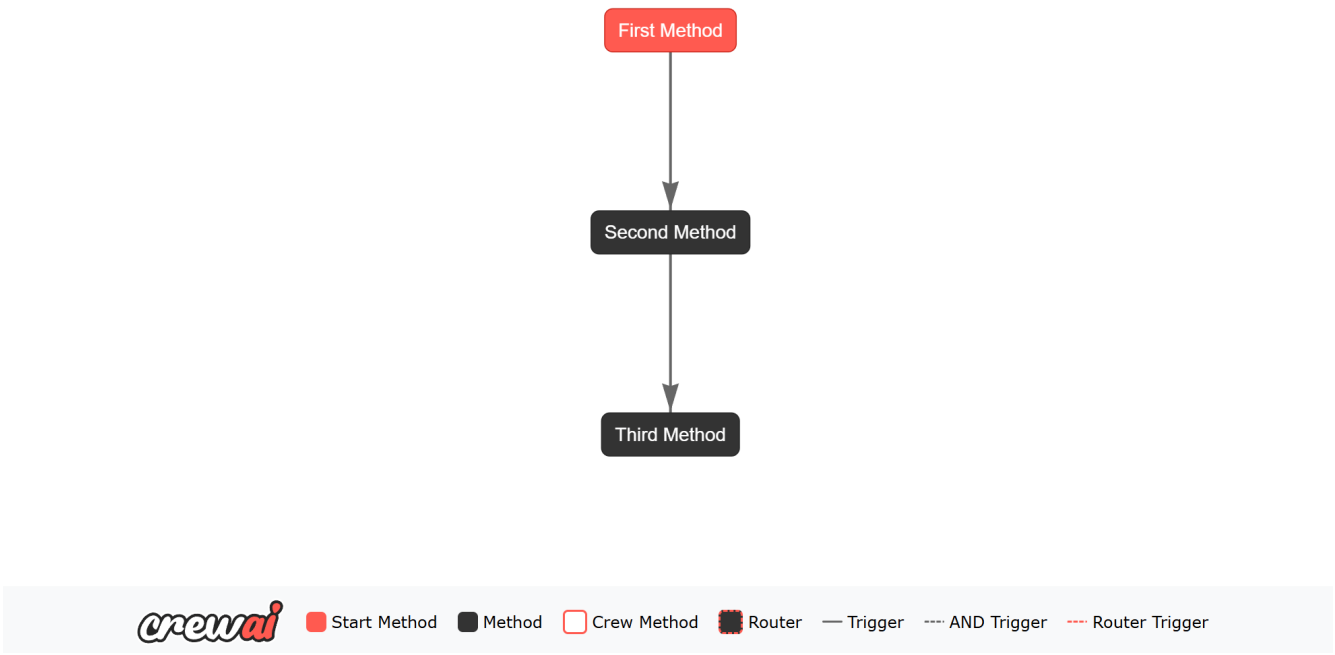


Figure 8: Flow Visual image

Key Points:

- **Defined Schema:** `ExampleState` clearly outlines the state structure, enhancing code readability and maintainability.
- **Type Safety:** Leveraging Pydantic ensures that state attributes adhere to the specified types, reducing runtime errors.
- **Auto-Completion:** IDEs can provide better auto-completion and error checking based on the defined state model.

Choosing Between Unstructured and Structured State Management

- **Use Unstructured State Management when:**
  - The workflow's state is simple or highly dynamic.
  - Flexibility is prioritized over strict state definitions.
  - Rapid prototyping is required without the overhead of defining schemas.
- **Use Structured State Management when:**
  - The workflow requires a well-defined and consistent state structure.
  - Type safety and validation are important for your application's reliability.
  - You want to leverage IDE features like auto-completion and type checking for better developer experience.

By providing both unstructured and structured state management options, CrewAI Flows empowers developers to build AI workflows that are both flexible and robust, catering to a wide range of application requirements.

Flow Persistence

The @persist decorator enables automatic state persistence in CrewAI Flows, allowing you to maintain flow state across restarts or different workflow executions. This decorator can be applied at either the class level or method level, providing flexibility in how you manage state persistence.

Class-Level Persistence

When applied at the class level, the @persist decorator automatically persists all flow method states:

```
@persist  # Using SQLiteFlowPersistence by default
class MyFlow(Flow[MyState]):
    @start()
    def initialize_flow(self):
        # This method will automatically have its state persisted
        self.state.counter = 1
        print("Initialized flow. State ID:", self.state.id)

    @listen(initialize_flow)
    def next_step(self):
        # The state (including self.state.id) is automatically reloaded
        self.state.counter += 1
        print("Flow state is persisted. Counter:", self.state.counter)
```

Method-Level Persistence

For more granular control, you can apply @persist to specific methods:

```
class AnotherFlow(Flow[dict]):
    @persist  # Persists only this method's state
    @start()
    def begin(self):
        if "runs" not in self.state:
            self.state["runs"] = 0
        self.state["runs"] += 1
        print("Method-level persisted runs:", self.state["runs"])
```

Forking Persisted State

`@persist` supports two distinct hydration modes on `kickoff` / `kickoff_async`:

- `kickoff(inputs={"id": <uuid>})` – **resume**: load the latest snapshot for the supplied UUID and continue writing under the same `flow_uuid`. The history extends.
- `kickoff(restore_from_state_id=<uuid>)` – **fork**: load the latest snapshot for the supplied UUID, hydrate the new run's state from it, and assign a fresh `state.id` (auto-generated, or `inputs["id"]` if pinned). The new run's `@persist` writes land under the new `state.id`; the source flow's history is preserved.

```
from crewai.flow.flow import Flow, start
from crewai.flow.persistence import persist
from pydantic import BaseModel

class CounterState(BaseModel):
    id: str = ""
    counter: int = 0

@persist
class CounterFlow(Flow[CounterState]):
```

```
    @start()
    def step(self):
        self.state.counter += 1
        print(f"[id={self.state.id}] counter={self.state.counter}")

# Run 1: fresh state, counter 0 -> 1, persisted under flow_1.state.id
flow_1 = CounterFlow()
flow_1.kickoff()

# Fork: hydrate from flow_1's latest snapshot, but use a NEW state.id
flow_2 = CounterFlow()
flow_2.kickoff(restore_from_state_id=flow_1.state.id)
# flow_2.state.counter starts at 1 (hydrated), then step() bumps it to 2.
# flow_2.state.id != flow_1.state.id; flow_1's history is unchanged.
```

If the supplied `restore_from_state_id` does not match any persisted state, the kickoff falls back silently – same as the existing `inputs["id"]` resume not-found behavior. Combining `restore_from_state_id` with `from_checkpoint` raises a `ValueError`; pick one hydration source. Pinning `inputs["id"]` while forking shares a persistence key with another flow – usually you want only `restore_from_state_id`.

How It Works

1. **Unique State Identification**
   - Each flow state automatically receives a unique UUID
   - The ID is preserved across state updates and method calls
   - Supports both structured (Pydantic BaseModel) and unstructured (dictionary) states
2. **Default SQLite Backend**
   - SQLiteFlowPersistence is the default storage backend
   - States are automatically saved to a local SQLite database
   - Robust error handling ensures clear messages if database operations fail
3. **Error Handling**
   - Comprehensive error messages for database operations
   - Automatic state validation during save and load
   - Clear feedback when persistence operations encounter issues

Important Considerations

- **State Types**: Both structured (Pydantic BaseModel) and unstructured (dictionary) states are supported
- **Automatic ID**: The `id` field is automatically added if not present
- **State Recovery**: Failed or restarted flows can automatically reload their previous state
- **Custom Implementation**: You can provide your own FlowPersistence implementation for specialized storage needs

Technical Advantages

1. **Precise Control Through Low-Level Access**
   - Direct access to persistence operations for advanced use cases
   - Fine-grained control via method-level persistence decorators
   - Built-in state inspection and debugging capabilities
   - Full visibility into state changes and persistence operations
2. **Enhanced Reliability**
   - Automatic state recovery after system failures or restarts
   - Transaction-based state updates for data integrity
   - Comprehensive error handling with clear error messages
   - Robust validation during state save and load operations
3. **Extensible Architecture**
   - Customizable persistence backend through FlowPersistence interface
   - Support for specialized storage solutions beyond SQLite

- Compatible with both structured (Pydantic) and unstructured (dict) states
- Seamless integration with existing CrewAI flow patterns

The persistence system's architecture emphasizes technical precision and customization options, allowing developers to maintain full control over state management while benefiting from built-in reliability features.

Flow Control

Conditional Logic: `or`

The `or_` function in Flows allows you to listen to multiple methods and trigger the listener method when any of the specified methods emit an output.

```
from crewai.flow.flow import Flow, listen, or_, start

class OrExampleFlow(Flow):

    @start()
    def start_method(self):
        return "Hello from the start method"

    @listen(start_method)
    def second_method(self):
        return "Hello from the second method"

    @listen(or_(start_method, second_method))
    def logger(self, result):
        print(f"Logger: {result}")



flow = OrExampleFlow()
flow.plot("my_flow_plot")
flow.kickoff()
```

```
Logger: Hello from the start method
Logger: Hello from the second method
```

When you run this Flow, the `logger` method will be triggered by the output of either the `start_method` or the `second_method`. The `or_` function is used to listen to multiple methods and trigger the listener method when any of the specified methods emit an output.

Conditional Logic: `and`

The `and_` function in Flows allows you to listen to multiple methods and trigger the listener method only when all the specified methods emit an output.

```
from crewai.flow.flow import Flow, and_, listen, start

class AndExampleFlow(Flow):

    @start()
    def start_method(self):
        self.state["greeting"] = "Hello from the start method"

    @listen(start_method)
    def second_method(self):
        self.state["joke"] = "What do computers eat? Microchips."

    @listen(and_(start_method, second_method))
```

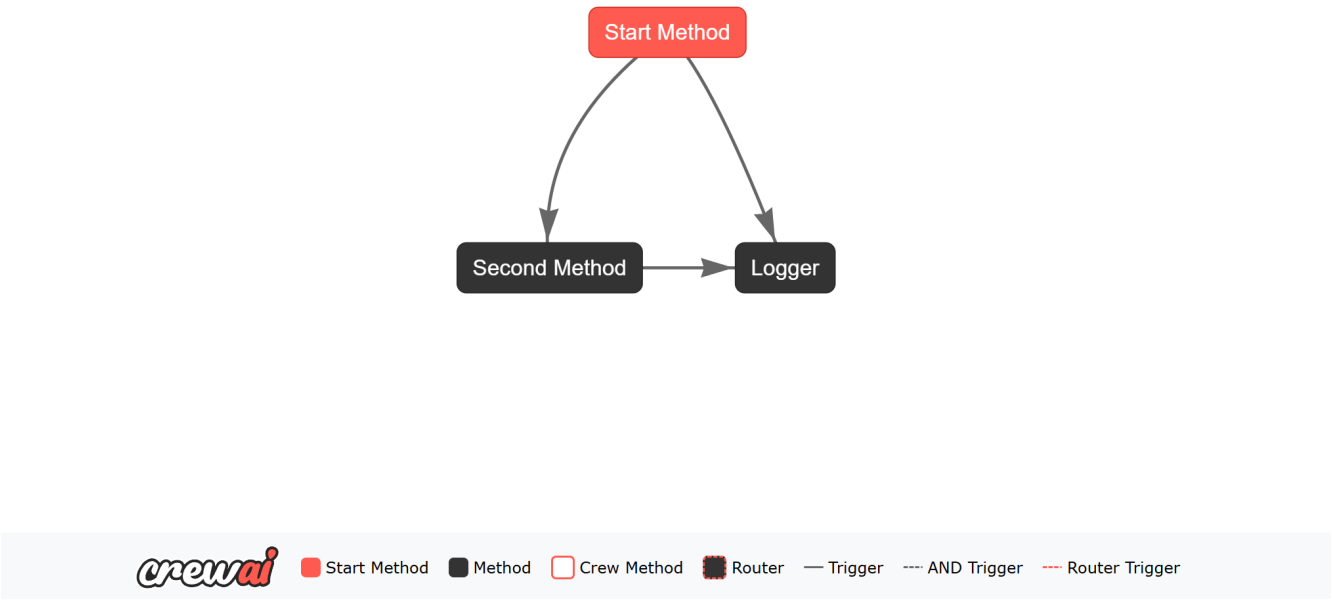


Figure 9: Flow Visual image

```
    def logger(self):
        print("---- Logger ----")
        print(self.state)

flow = AndExampleFlow()
flow.plot()
flow.kickoff()
```

```
---- Logger ----
{'greeting': 'Hello from the start method', 'joke': 'What do computers eat? Microchips.'}
```

When you run this Flow, the `logger` method will be triggered only when both the `start_method` and the `second_method` emit an output. The `and_` function is used to listen to multiple methods and trigger the listener method only when all the specified methods emit an output.

Router

The `@router()` decorator in Flows allows you to define conditional routing logic based on the output of a method. You can specify different routes based on the output of the method, allowing you to control the flow of execution dynamically.

```
import random
from typing import Literal

from crewai.flow.flow import Flow, listen, router, start
from pydantic import BaseModel

class ExampleState(BaseModel):
    success_flag: bool = False

class RouterFlow(Flow[ExampleState]):

    @start()
    def start_method(self):
```

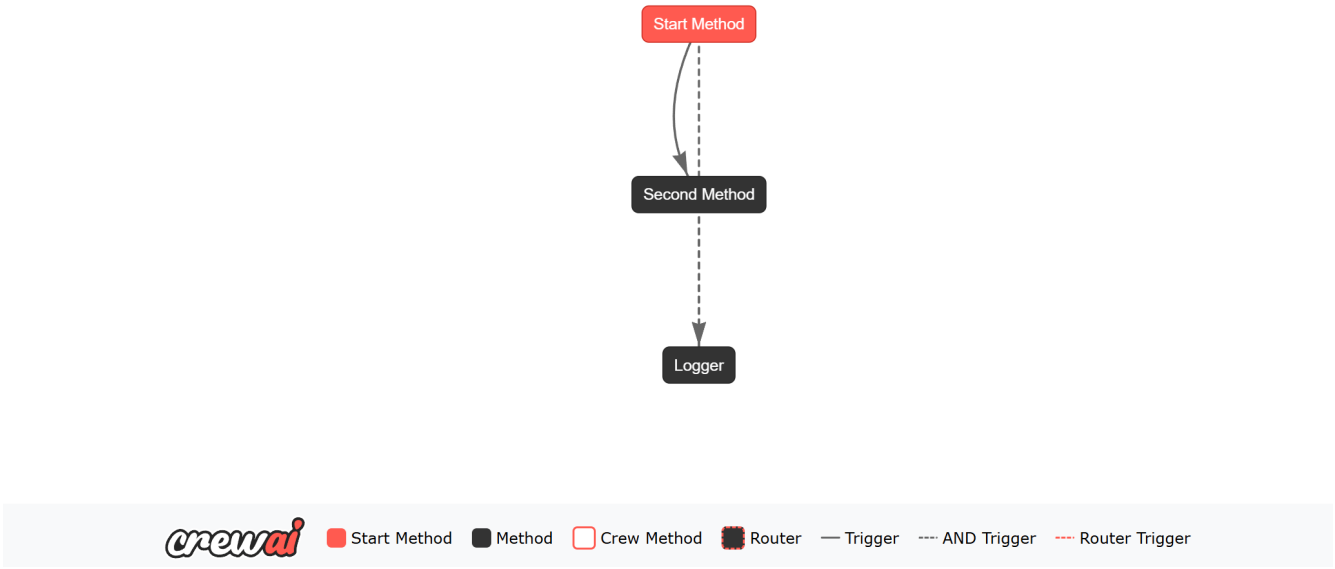


Figure 10: Flow Visual image

```
        print("Starting the structured flow")
        random_boolean = random.choice([True, False])
        self.state.success_flag = random_boolean

    @router(start_method)
    def second_method(self) -> Literal["success", "failed"]:
        if self.state.success_flag:
            return "success"
        else:
            return "failed"

    @listen("success")
    def third_method(self):
        print("Third method running")

    @listen("failed")
    def fourth_method(self):
        print("Fourth method running")


flow = RouterFlow()
flow.plot("my_flow_plot")
flow.kickoff()
```

```
Starting the structured flow
Third method running
Fourth method running
```

In the above example, the `start_method` generates a random boolean value and sets it in the state. The `second_method` uses the `@router()` decorator to define conditional routing logic based on the value of the boolean. If the boolean is `True`, the method returns `"success"`, and if it is `False`, the method returns `"failed"`. The `third_method` and `fourth_method` listen to the output of the `second_method` and execute based on the returned value.

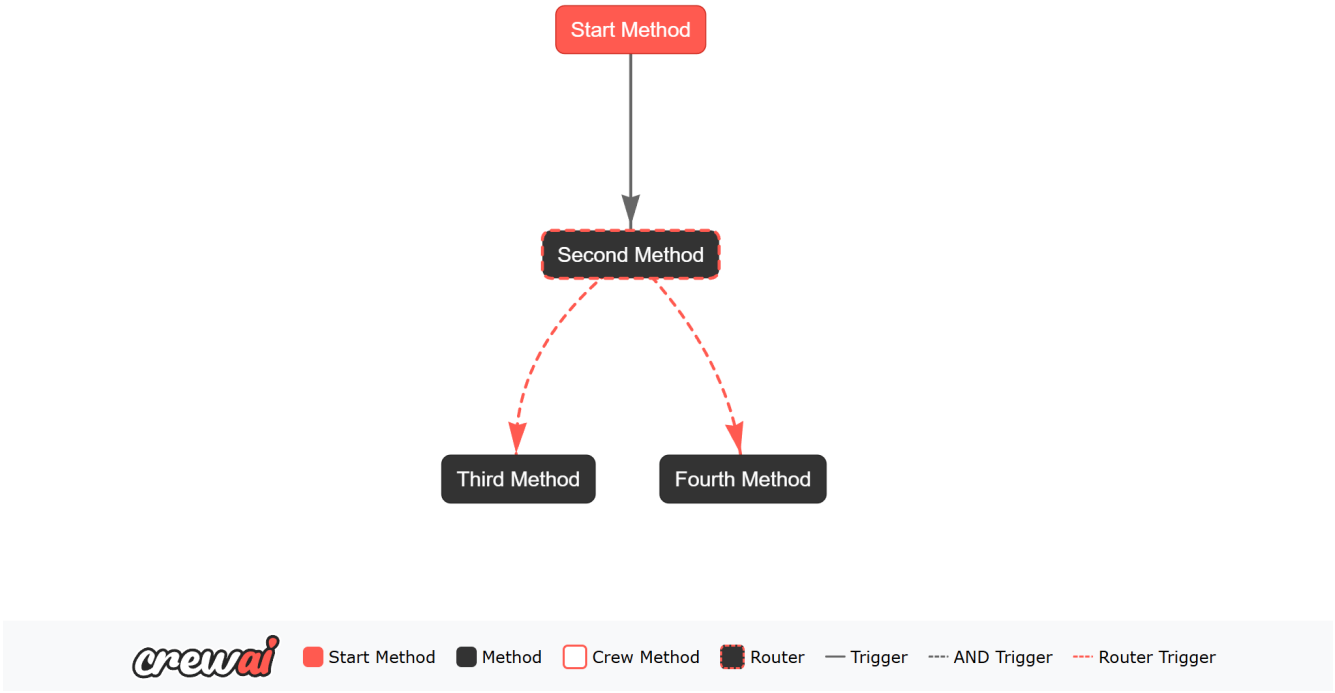


Figure 11: Flow Visual image

When you run this Flow, the output will change based on the random boolean value generated by the `start_method`.

Declare a router's labels so tools can see its branches: annotate the return type with `Literal[...]`, as `second_method` does, or pass them explicitly with `@router(start_method, emit=["success", "failed"])`. `flow.plot()` and `crewai flow plot` read the labels from there; without them, the plot can't draw the router's branches.

Human in the Loop (human feedback)

[!NOTE]

The `@human_feedback` decorator requires **CrewAI version 1.8.0 or higher**.

The `@human_feedback` decorator enables human-in-the-loop workflows by pausing flow execution to collect feedback from a human. This is useful for approval gates, quality review, and decision points that require human judgment.

```
from crewai.flow.flow import Flow, start, listen
from crewai.flow.human_feedback import human_feedback, HumanFeedbackResult

class ReviewFlow(Flow):
    @start()
    @human_feedback(
        message="Do you approve this content?",
        emit=["approved", "rejected", "needs_revision"],
        llm="gpt-4o-mini",
        default_outcome="needs_revision",
    )
    def generate_content(self):
        return "Content to be reviewed..."

    @listen("approved")
    def on_approval(self, result: HumanFeedbackResult):
        print(f"Approved! Feedback: {result.feedback}")
```

```
    @listen("rejected")
    def on_rejection(self, result: HumanFeedbackResult):
        print(f"Rejected. Reason: {result.feedback}")
```

When `emit` is specified, the human's free-form feedback is interpreted by an LLM and collapsed into one of the specified outcomes, which then triggers the corresponding `@listen` decorator.

You can also use `@human_feedback` without routing to simply collect feedback:

```
@start()
@human_feedback(message="Any comments on this output?")
def my_method(self):
    return "Output for review"

@listen(my_method)
def next_step(self, result: HumanFeedbackResult):
    # Access feedback via result.feedback
    # Access original output via result.output
    pass
```

Access all feedback collected during a flow via `self.last_human_feedback` (most recent) or `self.human_feedback_history` (all feedback as a list).

For a complete guide on human feedback in flows, including **async/non-blocking feedback** with custom providers (Slack, webhooks, etc.), see Human Feedback in Flows.

Adding Agents to Flows

Agents can be seamlessly integrated into your flows, providing a lightweight alternative to full Crews when you need simpler, focused task execution. Here's an example of how to use an Agent within a flow to perform market research:

```
import asyncio
from typing import Any, Dict, List

from crewai_tools import SerperDevTool
from pydantic import BaseModel, Field

from crewai.agent import Agent
from crewai.flow.flow import Flow, listen, start


# Define a structured output format
class MarketAnalysis(BaseModel):
    key_trends: List[str] = Field(description="List of identified market trends")
    market_size: str = Field(description="Estimated market size")
    competitors: List[str] = Field(description="Major competitors in the space")


# Define flow state
class MarketResearchState(BaseModel):
    product: str = ""
    analysis: MarketAnalysis | None = None


# Create a flow class
class MarketResearchFlow(Flow[MarketResearchState]):
    @start()
    def initialize_research(self) -> Dict[str, Any]:
        print(f"Starting market research for {self.state.product}")
```

```
    return {"product": self.state.product}

@listen(initialize_research)
async def analyze_market(self) -> Dict[str, Any]:
    # Create an Agent for market research
    analyst = Agent(
        role="Market Research Analyst",
        goal=f"Analyze the market for {self.state.product}",
        backstory="You are an experienced market analyst with expertise in "
        "identifying market trends and opportunities.",
        tools=[SerperDevTool()],
        verbose=True,
    )

    # Define the research query
    query = f"""
    Research the market for {self.state.product}. Include:
    1. Key market trends
    2. Market size
    3. Major competitors

    Format your response according to the specified structure.
    """

    # Execute the analysis with structured output format
    result = await analyst.kickoff_async(query, response_format=MarketAnalysis)
    if result.pydantic:
        print("result", result.pydantic)
    else:
        print("result", result)

    # Return the analysis to update the state
    return {"analysis": result.pydantic}

@listen(analyze_market)
def present_results(self, analysis) -> None:
    print("\nMarket Analysis Results")
    print("======================")

    if isinstance(analysis, dict):
        # If we got a dict with 'analysis' key, extract the actual analysis object
        market_analysis = analysis.get("analysis")
    else:
        market_analysis = analysis

    if market_analysis and isinstance(market_analysis, MarketAnalysis):
        print("\nKey Market Trends:")
        for trend in market_analysis.key_trends:
            print(f"- {trend}")

        print(f"\nMarket Size: {market_analysis.market_size}")

        print("\nMajor Competitors:")
        for competitor in market_analysis.competitors:
            print(f"- {competitor}")
    else:
```

```
            print("No structured analysis data available.")
            print("Raw analysis:", analysis)


# Usage example
async def run_flow():
    flow = MarketResearchFlow()
    flow.plot("MarketResearchFlowPlot")
    result = await flow.kickoff_async(inputs={"product": "AI-powered chatbots"})
    return result


# Run the flow
if __name__ == "__main__":
    asyncio.run(run_flow())
```

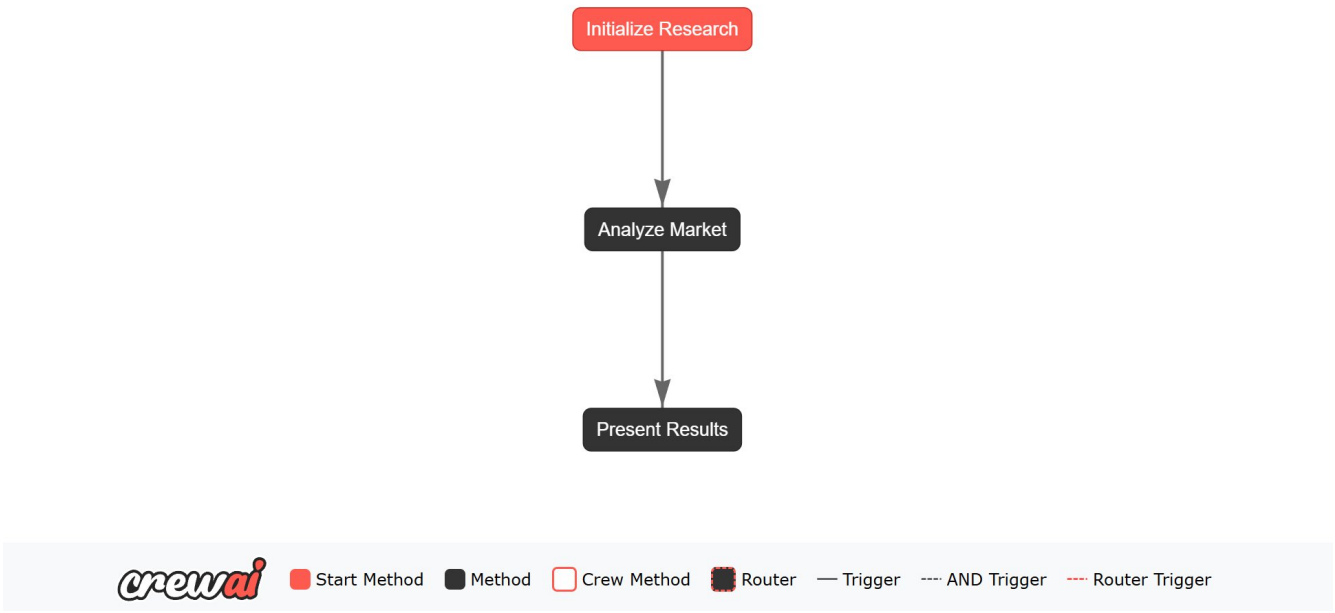


Figure 12: Flow Visual image

This example demonstrates several key features of using Agents in flows:

1. **Structured Output**: Using Pydantic models to define the expected output format (`MarketAnalysis`) ensures type safety and structured data throughout the flow.
2. **State Management**: The flow state (`MarketResearchState`) maintains context between steps and stores both inputs and outputs.
3. **Tool Integration**: Agents can use tools (like `WebsiteSearchTool`) to enhance their capabilities.

Adding Crews to Flows

Creating a flow with multiple crews in CrewAI is straightforward.

You can generate a new CrewAI project that includes all the scaffolding needed to create a flow with multiple crews by running the following command:

```
crewai create flow name_of_flow
```

This command will generate a new CrewAI project with the necessary folder structure. The generated project includes a prebuilt crew called `poem_crew` that is already working. The starter embedded crew uses the classic Python/YAML layout; new standalone crews created with `crewai create crew` use the JSON-first layout.

Folder Structure

After running the `crewai create flow name_of_flow` command, you will see a folder structure similar to the following:

Directory/File	Description
`name_of_flow/`	Root directory for the flow.
`crews/`	Contains directories for specific crews.
`poem_crew/`	Directory for the "poem_crew" with its configurations and scripts.
`config/`	Configuration files directory for the "poem_crew".
`agents.yaml`	YAML file defining the agents for "poem_crew".
`tasks.yaml`	YAML file defining the tasks for "poem_crew".
`poem_crew.py`	Script for "poem_crew" functionality.
`tools/`	Directory for additional tools used in the flow.
`custom_tool.py`	Custom tool implementation.
`main.py`	Main script for running the flow.
`README.md`	Project description and instructions.
`pyproject.toml`	Configuration file for project dependencies and settings.
`.gitignore`	Specifies files and directories to ignore in version control.

Building Your Crews

In the `crews` folder, you can define multiple crews. The generated `poem_crew` uses the classic embedded-crew structure:

- `config/agents.yaml`: Defines the agents for the crew.
- `config/tasks.yaml`: Defines the tasks for the crew.
- `poem_crew.py`: Contains the crew definition, including agents, tasks, and the crew itself.

You can copy, paste, and edit the `poem_crew` to create other classic embedded crews.

For JSON-first embedded crews, use a folder with `crew.jsonc` and `agents/*.jsonc` instead:

```
crews/
 research_crew/
     agents/
        researcher.jsonc
     crew.jsonc
```

Then load it from a Flow step:

```python
from pathlib import Path
from crewai.project import load_crew

crew, default_inputs = load_crew(
    Path(__file__).parent / "crews" / "research_crew" / "crew.jsonc"
)
result = crew.kickoff(inputs={**default_inputs, "topic": "AI Agents"})
```

Connecting Crews in `main.py`

The `main.py` file is where you create your flow and connect the crews together. You can define your flow by using the `Flow` class and the decorators `@start` and `@listen` to specify the flow of execution.

Here's an example of how you can connect the `poem_crew` in the `main.py` file:

```python
#!/usr/bin/env python
from random import randint

from pydantic import BaseModel
from crewai.flow.flow import Flow, listen, start
from .crews.poem_crew.poem_crew import PoemCrew

class PoemState(BaseModel):
    sentence_count: int = 1
    poem: str = ""

class PoemFlow(Flow[PoemState]):

    @start()
    def generate_sentence_count(self):
        print("Generating sentence count")
        self.state.sentence_count = randint(1, 5)

    @listen(generate_sentence_count)
    def generate_poem(self):
        print("Generating poem")
        result = PoemCrew().crew().kickoff(inputs={"sentence_count": self.state.sentence_count})

        print("Poem generated", result.raw)
        self.state.poem = result.raw

    @listen(generate_poem)
    def save_poem(self):
        print("Saving poem")
        with open("poem.txt", "w") as f:
            f.write(self.state.poem)

def kickoff():
    poem_flow = PoemFlow()
    poem_flow.kickoff()


def plot():
    poem_flow = PoemFlow()
    poem_flow.plot("PoemFlowPlot")

if __name__ == "__main__":
    kickoff()
    plot()
```

In this example, the `PoemFlow` class defines a flow that generates a sentence count, uses the `PoemCrew` to generate a poem, and then saves the poem to a file. The flow is kicked off by calling the `kickoff()` method. The PoemFlowPlot will be generated by `plot()` method.

Running the Flow

(Optional) Before running the flow, you can install the dependencies by running:

```
crewai install
```

Once all of the dependencies are installed, you need to activate the virtual environment by running:

```
source .venv/bin/activate
```

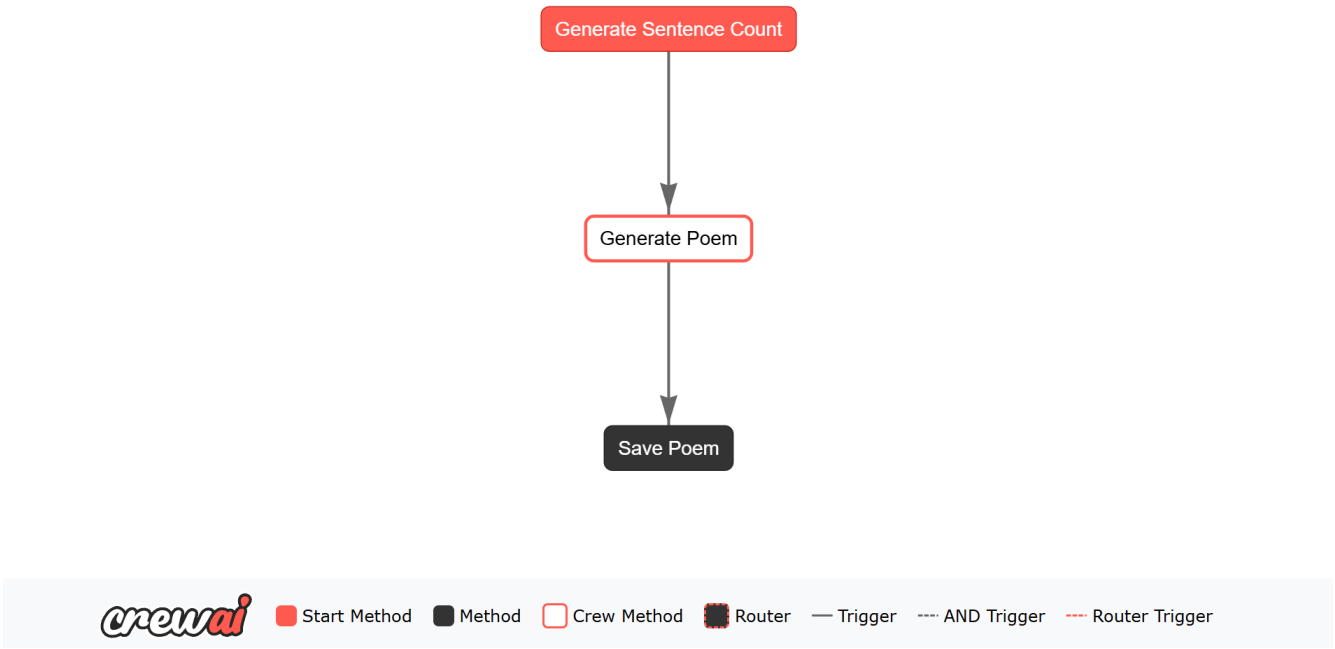


Figure 13: Flow Visual image

After activating the virtual environment, you can run the flow with the CrewAI CLI:

```
crewai run
```

You can also run the project script directly:

```
uv run kickoff
```

The flow will execute, and you should see the output in the console.

To run the flow with other inputs, pass them as a JSON object. `crewai run` kicks off the flow with them, the way a deployed flow is kicked off, and they set the matching state fields:

```
crewai run --inputs '{"topic": "Quantum Computing"}'
```

Plot Flows

Visualizing your AI workflows can provide valuable insights into the structure and execution paths of your flows. CrewAI offers a powerful visualization tool that allows you to generate interactive plots of your flows, making it easier to understand and optimize your AI workflows.

What are Plots?

Plots in CrewAI are graphical representations of your AI workflows. They display the various tasks, their connections, and the flow of data between them. This visualization helps in understanding the sequence of operations, identifying bottlenecks, and ensuring that the workflow logic aligns with your expectations.

How to Generate a Plot

CrewAI provides two convenient methods to generate plots of your flows:

Option 1: Using the `plot()` Method If you are working directly with a flow instance, you can generate a plot by calling the `plot()` method on your flow object. This method will create an HTML file containing the interactive plot of your flow.

```
# Assuming you have a flow instance
flow.plot("my_flow_plot")
```

This will generate a file named `my_flow_plot.html` in your current directory. You can open this file in a web browser to view the interactive plot.

Option 2: Using the Command Line If you are working within a structured CrewAI project, you can generate a plot using the command line. This is particularly useful for larger projects where you want to visualize the entire flow setup.

```
crewai flow plot
```

This command will generate an HTML file with the plot of your flow, similar to the `plot()` method. The file will be saved in your project directory, and you can open it in a web browser to explore the flow.

Understanding the Plot

The generated plot will display nodes representing the tasks in your flow, with directed edges indicating the flow of execution. The plot is interactive, allowing you to zoom in and out, and hover over nodes to see additional details.

By visualizing your flows, you can gain a clearer understanding of the workflow's structure, making it easier to debug, optimize, and communicate your AI processes to others.

Conclusion

Plotting your flows is a powerful feature of CrewAI that enhances your ability to design and manage complex AI workflows. Whether you choose to use the `plot()` method or the command line, generating plots will provide you with a visual representation of your workflows, aiding in both development and presentation.

Next Steps

If you're interested in exploring additional examples of flows, we have a variety of recommendations in our examples repository. Here are four specific flow examples, each showcasing unique use cases to help you match your current problem type to a specific example:

1. **Email Auto Responder Flow**: This example demonstrates an infinite loop where a background job continually runs to automate email responses. It's a great use case for tasks that need to be performed repeatedly without manual intervention. View Example
2. **Lead Score Flow**: This flow showcases adding human-in-the-loop feedback and handling different conditional branches using the router. It's an excellent example of how to incorporate dynamic decision-making and human oversight into your workflows. View Example
3. **Write a Book Flow**: This example excels at chaining multiple crews together, where the output of one crew is used by another. Specifically, one crew outlines an entire book, and another crew generates chapters based on the outline. Eventually, everything is connected to produce a complete book. This flow is perfect for complex, multi-step processes that require coordination between different tasks. View Example
4. **Meeting Assistant Flow**: This flow demonstrates how to broadcast one event to trigger multiple follow-up actions. For instance, after a meeting is completed, the flow can update a Trello board, send a Slack message, and save the results. It's a great example of handling multiple outcomes from a single event, making it ideal for comprehensive task management and notification systems. View Example

By exploring these examples, you can gain insights into how to leverage CrewAI Flows for various use cases, from automating repetitive tasks to managing complex, multi-step processes with dynamic decision-making and human feedback.

Also, check out our YouTube video on how to use flows in CrewAI below!

CrewAI Flows overview

Running Flows

There are two ways to run a flow:

Using the Flow API

You can run a flow programmatically by creating an instance of your flow class and calling the `kickoff()` method:

```
flow = ExampleFlow()
result = flow.kickoff()
```

Streaming Flow Execution

For real-time visibility into flow execution, you can enable streaming to receive output as it's generated:

```
class StreamingFlow(Flow):
    stream = True  # Enable streaming

    @start()
    def research(self):
        # Your flow implementation
        pass

# Iterate over streaming output
flow = StreamingFlow()
streaming = flow.kickoff()
for chunk in streaming:
    print(chunk.content, end="", flush=True)

# Access final result
result = streaming.result
```

Learn more about streaming in the Streaming Flow Execution guide.

Memory in Flows

Every Flow automatically has access to CrewAI's unified Memory system. You can store, recall, and extract memories directly inside any flow method using three built-in convenience methods.

Built-in Methods

Method	Description
`self.remember(content, **kwargs)`	Store content in memory. Accepts optional `scope`, `categories`, `metadata`, `importance`.
`self.recall(query, **kwargs)`	Retrieve relevant memories. Accepts optional `scope`, `categories`, `limit`, `depth`.
`self.extract_memories(content)`	Break raw text into discrete, self-contained memory statements.

A default `Memory()` instance is created automatically when the Flow initializes. You can also pass a custom one:

```
from crewai.flow.flow import Flow
from crewai import Memory

custom_memory = Memory(
    recency_weight=0.5,
    recency_half_life_days=7,
    embedder={"provider": "ollama", "config": {"model_name": "mxbai-embed-large"}},
```

```
)

flow = MyFlow(memory=custom_memory)
```

Example: Research and Analyze Flow

```
from crewai.flow.flow import Flow, listen, start


class ResearchAnalysisFlow(Flow):
    @start()
    def gather_data(self):
        # Simulate research findings
        findings = (
            "PostgreSQL handles 10k concurrent connections with connection pooling. "
            "MySQL caps at around 5k. MongoDB scales horizontally but adds complexity."
        )

        # Extract atomic facts and remember each one
        memories = self.extract_memories(findings)
        for mem in memories:
            self.remember(mem, scope="/research/databases")

        return findings

    @listen(gather_data)
    def analyze(self, raw_findings):
        # Recall relevant past research (from this run or previous runs)
        past = self.recall("database performance and scaling", limit=10, depth="shallow")

        context_lines = [f"- {m.record.content}" for m in past]
        context = "\n".join(context_lines) if context_lines else "No prior context."

        return {
            "new_findings": raw_findings,
            "prior_context": context,
            "total_memories": len(past),
        }


flow = ResearchAnalysisFlow()
result = flow.kickoff()
print(result)
```

Because memory persists across runs (backed by LanceDB on disk), the `analyze` step will recall findings from previous executions too – enabling flows that learn and accumulate knowledge over time.

See the Memory documentation for details on scopes, slices, composite scoring, embedder configuration, and more.

Using the CLI

Starting from version 0.103.0, you can run flows using the `crewai run` command:

```
crewai run
```

This command automatically detects if your project is a flow (based on the `type = "flow"` setting in your pyproject.toml) and runs it accordingly. This is the recommended way to run flows from the command line.

The legacy `crewai flow kickoff` command is deprecated. Use `crewai run` for both crews and flows.

This documentation is built and hosted on Mintlify, a developer documentation platform.